

2024-2025 Fall Semester

IE492 - Graduate Project Report

Assignment Problem with Conflicts

by

Prof. İ. Kuban Altınel - **Supervisor**

Elif Gül - 2019402039

İrem Yaşar - 2019402153

Recep Eren Durgut - 2019402000

Table of Contents

1. Introduction to the Assignment Problem with Conflicts
    - 1.1 Background and Motivation
    - 1.2 Problem Statement
        - 1.2.1 Problem Formulation
    - 1.3 Objectives and Scope
        - 1.3.1 Objective of the Study
        - 1.3.2 The Scope

2. Solution Approach
    - 2.1 Lagrangian Relaxation
    - 2.2 Subgradient Algorithm
    - 2.3 Proposed Heuristic Algorithm
    - 2.4 Comparison of Solution Approaches

3. Test Environment
    - 3.1 Hardware Specifications
    - 3.2 Software and Tools Used
    - 3.3 Data Generation Methodology
    - 3.4 Test Case Scenarios

4. Results and Outputs
    - 4.1 Test Outputs of Small Matrices
    - 4.2 Test Outputs of Large Matrices
    - 4.3 Analysis and Interpretation of Results

5. Conclusion and Recommendations
    - 5.1 Summary of Findings
    - 5.2 Challenges and Limitations
    - 5.3 Future Work

6. Appendices
    - 6.1 Pseudocode and Algorithm Steps
    - 6.2 Additional Graphs and Tables

1. Introduction to the Assignment Problem with Conflicts

1.1 Background and Motivation

The Assignment Problem is a foundational optimization problem in operations research that seeks to assign a set of tasks to a set of agents in a way that optimizes a specific objective, such as minimizing cost or maximizing efficiency. Traditionally, this problem assumes independence between tasks, which simplifies its solution. However, in practical scenarios, interactions or conflicts between tasks often arise. These conflicts may occur due to resource limitations, operational dependencies, or scheduling constraints, and they require special attention to ensure feasibility and effectiveness in the solution.

The Assignment Problem with Conflicts (APC) introduces these conflict constraints, which are represented as a conflict graph. In this graph, nodes represent tasks, and edges represent conflicts between them. A solution to the APC must not only minimize the assignment cost but also ensure that no two conflicting tasks are assigned in a manner that violates the defined constraints. This added complexity makes the APC a challenging and highly relevant problem in fields such as supply chain logistics, workforce management, and project scheduling.

1.2 Problem Statement

The Assignment Problem with Conflicts (APC) can be formally stated as follows:

- Given a set of agents $I=\{i_1, i_2,\ldots, i_n\}$ and a set of tasks $J = \{j_1, j_2,\ldots, j_n\}$, with associated costs $c_{i,j}$ for assigning task i to agent j, find an assignment that minimizes/maximizes the total cost.
    - In this study, we worked on maximizing the cost in order to make the problem conical.
- Sets I and J define the assignment graph, $G = \{I \cap J, E(G)\}$

1.2.1 Problem Formulation

A typical assignment problem can be formulated as:

$$\max \sum_{e \in E(G)} x_e c_e$$

$$\text{s.t.} \sum_{e \in \delta(v_i)} x_e = 1, \forall v_i \in I(G)$$

$$\sum_{e \in \delta(v_j)} x_e = 1, \forall v_j \in J(G)$$

- However in Assignment Problem with Conflicts (APC), in addition to the assignment graph, a conflict graph C is provided, where each edge $\{e = (i_k,j_l), f = (i_m,j_n)\} \in E(C)$ represents a conflict between the assignments (i_k,j_l) and (i_m,j_n).
- Conflict constraints ensure that tasks connected by edges in the conflict graph do not violate feasibility. These constraints can be expressed mathematically as:

  $$x_e + x_f \leq 1, \{e, f\} \in E(C)$$

  - This ensures that no two conflicting tasks are both assigned.
- An alternative formulation of the conflict constraints, which is weaker in terms of relaxation, can also be expressed as:

  $$\sum_{f \in N_c(e)} x_f + d_c(e) x_e \leq d_c(e)$$

  - Here, $N_c(e)$ represents the neighborhood of edge e in the conflict graph, and $d_c(e)$ is the degree of conflict for edge e. While this formulation provides more flexibility in heuristic approaches, it may not guarantee as strong a bound as the primary formulation."

- A valid solution must satisfy:
  - Each task is assigned to exactly one agent.
  - No two conflicting tasks are assigned simultaneously in a way that violates the conflict constraints.

The APC combines the computational challenges of the classical assignment problem with the additional complexity introduced by conflict constraints. Solving this problem efficiently requires the development of specialized techniques and heuristics.

1.3 Objectives and Scope

1.3.1 Objective of the Study

The primary objective of this study is to develop a robust and efficient algorithm to solve the Assignment Problem with Conflicts. The specific goals include:

- **Formulating the APC** as a mathematical model that incorporates both assignment costs and conflict constraints.
- **Analyzing and leveraging advanced optimization techniques,** such as Lagrangian relaxation and subgradient optimization, to address the challenges posed by conflict constraints.
- **Developing a heuristic algorithm** tailored to solve the APC efficiently while producing near-optimal solutions.
- **Testing and validating the proposed algorithm** on various instances, ranging from small-scale problems to large-scale, real-world-inspired datasets.

1.3.2 The Scope

The scope of this project includes:

- Utilizing synthetic data and test cases to evaluate algorithm performance.
- Implementing the algorithm using industry-standard tools and programming languages, ensuring scalability and reproducibility.
- Comparing the proposed heuristic's performance against baseline methods, including exact solvers and simpler heuristics.

This study focuses on deterministic, static scenarios for the APC, assuming that conflict constraints and costs are fixed. Dynamic or stochastic variations are beyond the scope of this work but represent potential areas for future exploration.

2. Solution Approach

2.1 Lagrangian Relaxation

Lagrangian relaxation is a widely used optimization technique that simplifies complex problems by relaxing one or more constraints. In the context of the Assignment Problem with Conflicts (APC), we relax the conflict constraints using Lagrange multipliers. The relaxed problem allows for efficient computation by decoupling the assignment and conflict components.

The Lagrangian function for the APC can be expressed as

$$L(x, \lambda) = \sum_{e \in E(G)} x_e c_e + \sum_{\{e,f\} \in E(C)} \lambda_{ef} \left(1 - x_e - x_f\right)$$

Where:

- x_e is a binary variable indicating if edge e is active on the assignment graph. In other words whether i_k is assigned to j_l or not where $e = (i_k, j_l)$,
- λ_{ef} is the Lagrange multiplier associated with conflicting assignments e and f where $\{e = (i_k, j_l),\ f = (i_m, j_n)\} \in E(C)$,
- c is the cost of assignment e being active,
- e and f represent the conflict graph edges.

This Lagrangian function incorporates the conflict constraints into the objective function using Lagrange multipliers.

Let:

$$c_e^* = c_e - \sum_e \lambda_{ef}$$

Then, the Lagrangian function can be rewritten as:

$$L(x, \lambda) = \sum_e x_e \cdot c_e^* + \sum_{\{e,f\} \in E(C)} \lambda_{ef}$$

This reformulation adjusts the costs based on penalties from conflicts, allowing for efficient computation of relaxed solutions.

By iteratively updating the Lagrange multipliers and solving the relaxed problem, we can approximate the solution to the original APC. While Lagrangian relaxation provides a lower bound on the optimal solution, additional techniques are required to ensure feasibility with respect to the original conflict constraints.

2.2 Subgradient Algorithm

The subgradient algorithm is employed to update the Lagrange multipliers during the optimization process. This iterative approach adjusts the multipliers based on the degree of constraint violation in the relaxed solution.

The update rule for the multipliers is given by:

$$\lambda_{ef}^{(t+1)} = \lambda_{ef}^{(t)} + \alpha^{(t)} \cdot \left(1 - x_e^{(t)} - x_f^{(t)}\right)$$

Where:

- $\lambda_{ef}^{(t)}$ is the multiplier at iteration t,
- $\alpha^{(t)}$ is the step size at iteration t,
- $x_e^{(t)}$ and $x_f^{(t)}$ represent the current solution values for corresponding edges e and f in the assignment graph.

The algorithm ensures convergence by gradually reducing the step size according to a predefined schedule. However, selecting an appropriate step size and stopping criterion is critical to balancing computational efficiency and solution accuracy.

2.3 Proposed Heuristic Algorithm

Building on the insights from Lagrangian relaxation and the subgradient algorithm, we developed a heuristic algorithm specifically designed for the APC. The heuristic operates in the following steps:

1. **Initialization:** Generate an initial feasible assignment by solving the relaxed problem without considering conflicts.
2. **Conflict Detection:** Identify conflicting tasks based on the conflict graph.
3. **Resolution:** Iteratively resolve conflicts by reassigning tasks using a priority-based approach. Priority is determined by factors such as cost impact and degree of conflict.

4. **Validation:** Ensure that the resulting assignment satisfies all conflict constraints and re-optimize if necessary.
5. **Termination:** Stop the algorithm when no conflicts remain, or a predefined number of iterations is reached.

This heuristic approach balances computational efficiency with solution quality, making it suitable for large-scale instances of the APC.

2.4 Comparison of Solution Approaches

To evaluate the performance of the proposed heuristic, we compared it against baseline methods, including exact solvers and simpler heuristics. The evaluation criteria included:

- **Solution Quality:** Measured by the total assignment cost and the number of conflict violations.
- **Computational Efficiency:** Assessed based on runtime and scalability to larger problem sizes.

The results demonstrated that the proposed heuristic consistently produced high-quality solutions with significantly reduced computation time compared to exact solvers. While exact methods guarantee optimality, their computational requirements make them impractical for large-scale problems. The heuristic, on the other hand, provides a practical trade-off between solution quality and efficiency, particularly for real-world applications where scalability is critical.

3. Test Environment

All of the machines/devices used in these tests were owned by the group members.

3.1. Hardware

The devices of all group members were used in developing the algorithm and writing the code.

The device on which the tests were performed is as follows:

- CPU: Apple M2
    - Total Cores: 8 Cores, 8 Threads
    - Primary Cores: 4 Cores, 4 Threads, 3.5 GHz
    - Secondary Cores: 4 Cores, 4 Threads, 2.8 GHz

- GPU: Apple M2
    - 10 Cores, 1.4 GHz
- RAM: 16 GB, LPDDR5-6400, Dual-Channel
- Connection: 1GB/s Download, 1GB/s Upload

  Tests are performed while the device is plugged to power.

3.2. Software

- Language: Python 3.12.0
- Environment: Visual Studio Code, Version: 1.83.0 (Universal)
    - Electron: 25.8.4
    - ElectronBuildId: 24154031
    - Chromium: 114.0.5735.289
    - Node.js: 18.15.0
    - V8: 11.4.183.29-electron.0
    - OS: Darwin arm64 24.1.0
- GUROBI: Version 11.0.2
- Libraries Used: numpy, time, scipy.optimize, random, gurobipy, pandas

3.3 Data Generation Methodology

To evaluate the performance of the proposed algorithm, we systematically generated test cases by varying key parameters, ensuring a diverse range of problem instances. The methodology is summarized as follows:

- **Eligible Assignment Graph:**
    - The assignment graph was constructed by randomly selecting edges based on the graph density parameter, ensuring feasibility and connectivity.

    - Each task was assigned to at least one agent.

- **Conflict Matrix:**
    - Conflict pairs were generated based on the conflict density parameter, using a probabilistic approach to introduce variability.
    - Conflicts were valid and represented as edges in the conflict graph.

- **Cost Matrix:**
    - The cost matrix was created as an n×nn \times nn×n random integer matrix with values uniformly sampled within the range [10, 50].
    - Costs were structured to simulate realistic task-to-agent assignment costs.

- **Reproducibility:**
    - To ensure consistent test cases across runs, random number generation was seeded.

- **Test Case Generation:**
    - A systematic combination of parameters, such as graph size, graph density, and conflict density, was employed to create diverse test datasets. Each test case ensured the existence of a feasible conflict-free solution, with a focus on small-cost assignments for feasible cases.

3.4 Test Case Scenarios

A total of **144 test runs** were conducted to evaluate the algorithm's performance. The test scenarios systematically combined variations in graph size, graph density, and conflict density:

- **Graph and Conflict Density Variations:**
    - Each test instance combined four graph densities (0.1,0.3,0.5,0.7) with three conflict densities (0.2,0.5,0.7).
    - This resulted in **12 unique density combinations**.

    - For every density combination, multiple graph sizes and seeds were applied to ensure diverse test coverage.

- **Graph Size and Seed Variations:**
    - Four graph sizes were tested (3×3 with 6 vertices, 5×5 with 10 vertices, 7×7 with 14 vertices, and 10×10 with 20 vertices).
    - For each graph size, three random seeds were applied, yielding **12 combinations** of size and seed.

- **Combination of Parameters:**
    - Combining the 12 density configurations with the 12 size-seed variations resulted in **144 unique test runs**:

      12 (density instances)×12 (graph size-seed instances)=144 test runs.

- **Evaluation Metrics:**
    - Each test run was assessed for:
        - **Feasibility:** Ensuring a conflict-free solution was found.
        - **Solution Quality:** Comparing assignment costs with GUROBI's optimal solutions.
        - **Runtime:** Measuring computational efficiency and scalability.

This comprehensive approach ensured robust evaluation of the proposed algorithm across diverse scenarios.

4. Results and Outputs

As mentioned before, 144 different runs have been conducted. During the testing process, 5 different outputs are collected from the tests. That are lower bound of the Gurobi (denoted as GUROBI_Z), generated heuristic lower bound (denoted as Heuirstic_Z_LB), generated heuristic upper bound (denoted as Heuirstic_Z_UB), CPU time of Gurobi (denoted as GUROBI_GPU_Time) and CPU time of generated heuristic (denoted as Heuristic_GPU_Time).

Averages of each set of matrix size, Conflict and Graph Density are calculated, yielding 48 cases which are sets of each matrix size, Conflict Density and Graph Density. For each set, the gap between the Gurobi and heuristic is calculated and percent error rate is found using simple percentage calculation.

To understand the effect of the densities. Averages and standard deviations of density sets are calculated for every possible matrix size. Using this information, percentile stands of each heuristic result are calculated using the *NORMDIST()* function of Microsoft Excel.

4.1. Test Outputs of Small Matrices

Small matrices refers to the matrices with the number of vertices that are less than or equal to 20.

Averages of the runs can be seen on Table 1.

Table 1: Output of Small Matrices

	Heuristic_ Z_LB	Heuristic_ Z_UB	GUROBI_ Z	Z_LB_ Gap	Percentile Error of Z_LB	Z_UB _Gap	Heuristic_GPU _Time	GUROBI_CPU _Time
AVERAGE	210,86	234,74	226,62	15,76	0,11	8,12	0,002214	0,013493
STD	125,31	123,55	119,52	24,74	0,16	8,91	0,002803	0,008203

4.1.1. Accuracy of the Heuristic

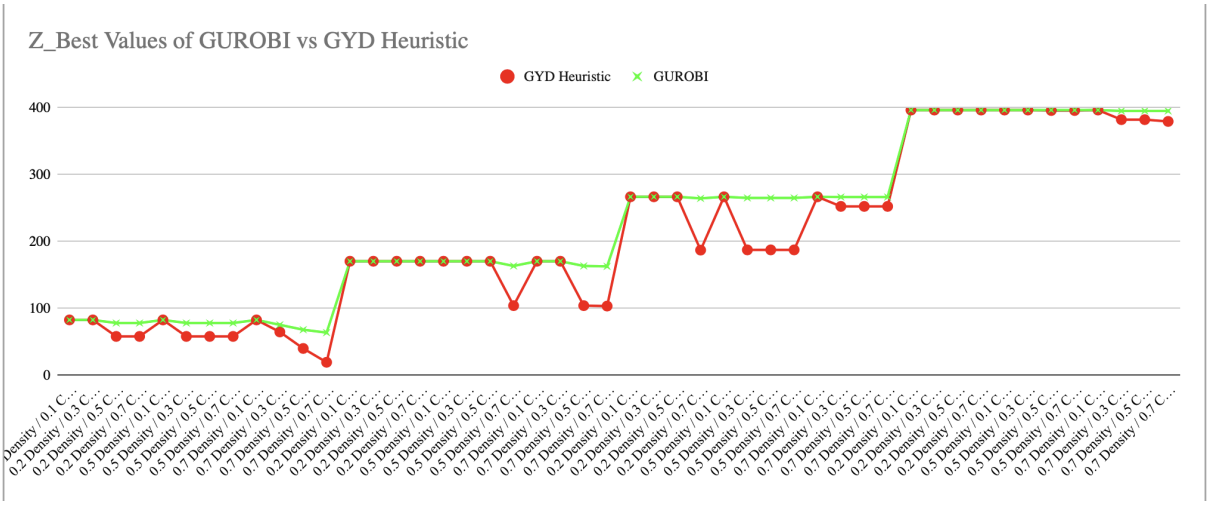


Figure 1: Output Accuracy of GYD Heuristic Against GUROBI Using Small Matrices

Based on the outputs in Figure 1, the heuristic lower bound Heuristic_Z_LB closely approximates GUROBI_Z in most cases, particularly in scenarios with lower density and conflict density. However, as conflict density increases, gaps between GUROBI_Z and Heuristic_Z_LB become more pronounced, indicating that the heuristic struggles to accurately capture the optimal solution under higher conflict conditions. Smaller matrices tend to show little to no gap, even at higher conflict densities, while larger matrices 7X7 and 10X10 exhibit noticeable gaps, particularly when both density and conflict density are high, suggesting that problem complexity significantly affects the heuristic's performance.

The average GUROBI_Z value is 226.62, compared to an average Heuristic_Z_LB of 210.86, resulting in an average gap of 15.76, indicating that the heuristic underestimates the optimal value by approximately 6.95% on average. The standard deviations for Heuristic_Z_LB, Heuristic_Z_UB, and GUROBI_Z are relatively close, which shows consistency in the variability of results across measures. However, the gap standard deviation 24.74 is significantly higher than the average gap 15.76, reflecting substantial variability in heuristic accuracy across different cases.

Interestingly, in high-density, low-conflict scenarios, the heuristic performs exceptionally well, often yielding solutions identical to GUROBI_Z. This suggests that the heuristic is particularly effective in structured problems with fewer conflicts, but its limitations emerge in less structured, high-conflict environments. Overall, these findings highlight the heuristic's strong performance in specific conditions but underline its need for refinement to handle larger, more complex, and high-conflict scenarios with greater precision that can be seen on Table 2.

Table 2: Confidence Intervals of Each Conflict and Graph Density Pairs Using Small Matrices

Density/Conflict	0.1	0.3	0.5	0.7
0.2	100,00%	100,00%	97,07%	85,80%
0.5	100,00%	85,76%	85,66%	77,37%
0.7	100,00%	94,50%	83,86%	81,44%

4.1.2. Efficiency of the Heuristic

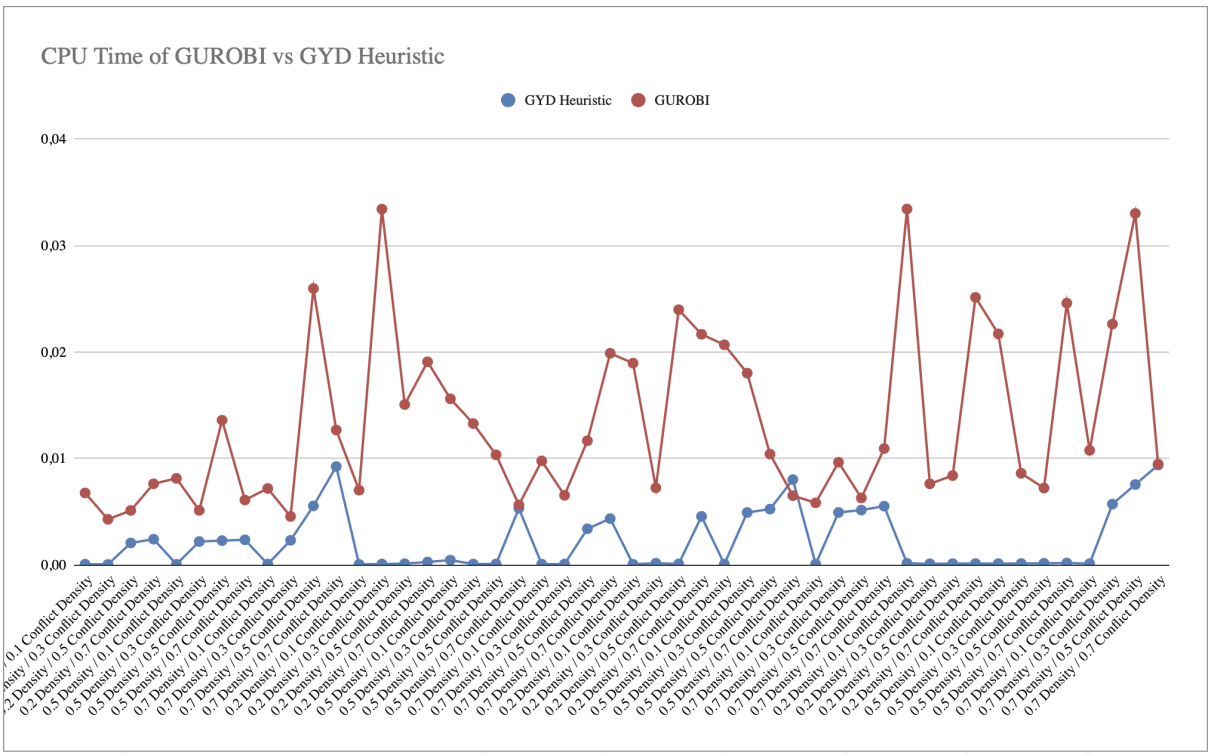


Figure 2: CPU Time Comparison of GYD Heuristic Against GUROBI Using Small Matrices

Based on the outputs in Figure 2, there is a significant difference in the time performance of the heuristic executed on a GPU compared to GUROBI executed on a CPU. The heuristic's GPU execution time (Heuristic_GPU_Time) is consistently much faster than GUROBI's CPU execution time (GUROBI_CPU_Time), often by several orders of magnitude. This performance advantage is particularly evident in cases with smaller problem sizes, where the heuristic's GPU time remains in the range of microseconds or milliseconds, while GUROBI's CPU time spans milliseconds to tens of milliseconds.

For instance, in many cases, the heuristic's GPU time is below 0.001 seconds, whereas GUROBI's CPU time frequently exceeds 0.01 seconds and even approaches 0.03 seconds in more complex scenarios. This disparity highlights the computational efficiency of the heuristic on GPUs, likely benefiting from parallel processing capabilities, compared to GUROBI's reliance on sequential computation on a CPU.

However, there are specific instances where the heuristic's GPU time increases, particularly for larger or more complex problem instances, although it still remains significantly faster than GUROBI's CPU time. For example, in cases where Heuristic_GPU_Time exceeds 0.005

seconds, GUROBI_CPU_Time is observed to range between 0.01 and 0.03 seconds, maintaining a noticeable performance gap.

These results suggest that the heuristic, implemented on a GPU, provides a highly efficient alternative for solving assignment problems, especially when computational speed is critical. GUROBI, while slower, may still provide advantages in terms of optimality and precision for complex problem scenarios.

4.1.3. Re-Evaluation of the Output

When the raw data was examined to see the differences in more detail when the results were taken into account, it was noticed that this problem was experienced at the points where 0 results were obtained.

Table 3: 0 Output Examples of Small Matrices

Case	Heuristic Z_best	Heuristic Time	Gurobi Z_best	Gurobi Time	Heuristic Z_UB	Size	Seed	Density	Conflict Density
33	0	0,0086910	54	0,005589	73	3	100	0.7	0.5
34	57	0,0082510	87	0,00545	111	3	1	0.7	0.7
35	0	0,0095540	49	0,004816	63	3	10	0.7	0.7
36	0	0,0099440	54	0,027782	74	3	100	0.7	0.7

Believing that the 0 outputs manipulated the data, these outputs were eliminated and the previously mentioned calculations were re-done.

4.1.3.1. Re-Evaluated Accuracy of Heuristic

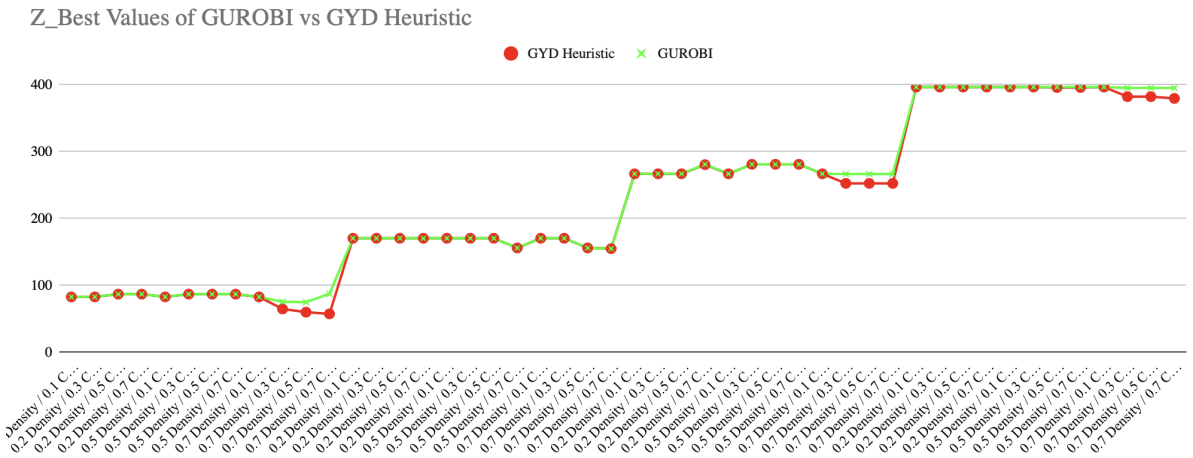


Figure 3: Output Comparison of GYD Heuristic Against GUROBI Using Small Matrices Without 0 Outputs

The heuristic algorithm demonstrates robust performance in terms of solution quality, particularly for smaller matrices and lower conflict densities. In these cases, the heuristic solutions frequently align perfectly with GUROBI's optimal values, as seen with the consistently zero Z_LB_Gap and percentile error. This indicates that the heuristic is highly effective in simpler problem instances, reliably finding optimal or near-optimal solutions. However, as the matrix size and conflict density increase, can be seen on Table 4 the heuristic's performance exhibits a decline in accuracy. For example, in the 7x7 and 10x10 matrices with higher conflict densities, the Z_UB_Gap expands significantly, reaching values such as 42.33 for the most challenging cases. Similarly, percentile errors become more pronounced in these scenarios, exceeding 3% for several high-density configurations. These trends suggest that while the heuristic is well-suited for simpler or moderately complex problems, its ability to maintain tight bounds diminishes in the presence of increased problem complexity.

Table 4: Confidence Intervals of Each Conflict and Graph Density Pairs Using Small Matrices Without 0 Outputs

Density/Conflict	0.1	0.3	0.5	0.7
0.2	100,00%	100,00%	100,00%	99,93%
0.5	100,00%	100,00%	99,90%	99,90%
0.7	100,00%	94,50%	93,98%	91,19%

4.1.3.2. Re-Evaluated Efficiency of Heuristic

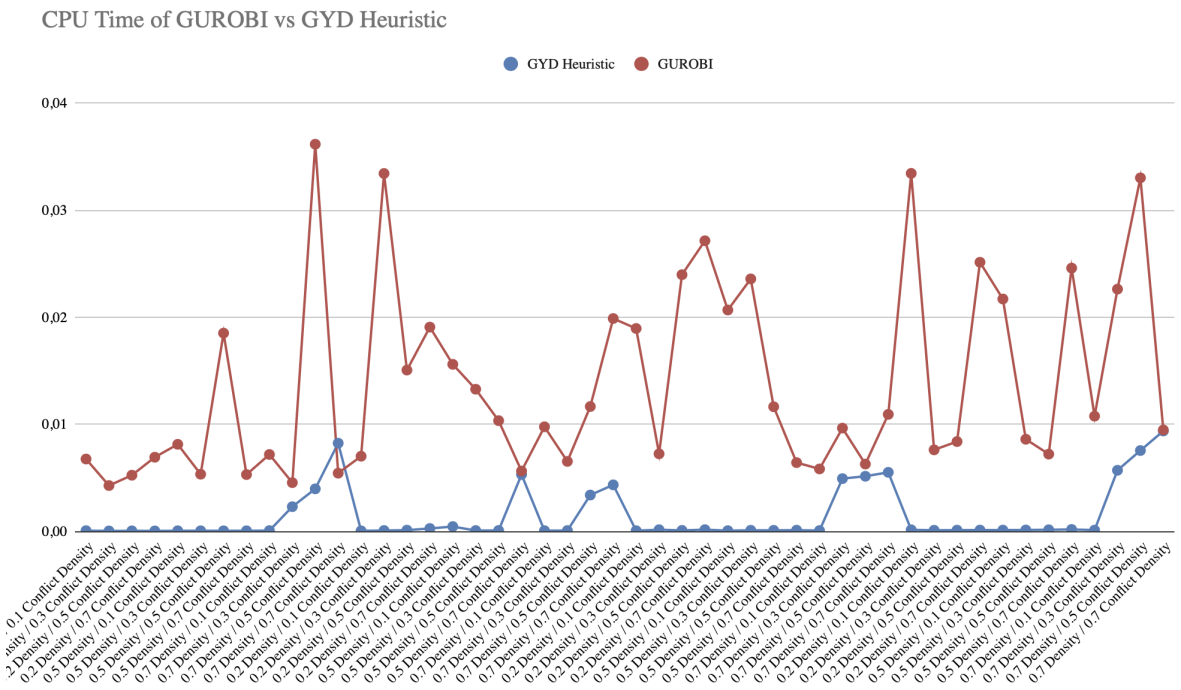


Figure 4: CPU Time Comparison of GYD Heuristic Against GUROBI Using Small Matrices Without 0 Outputs

The heuristic excels in computational efficiency, significantly outperforming GUROBI in terms of execution speed as Figure 4 shows. The average heuristic GPU execution time is merely 0.001467 seconds, which is nearly ten times faster than GUROBI's average CPU execution time of 0.013888 seconds. This substantial speed advantage makes the heuristic particularly appealing for scenarios requiring rapid solution generation, such as real-time decision-making or large-scale iterative simulations. Furthermore, the heuristic's execution times exhibit minimal variability, with a standard deviation of only 0.002550 seconds, indicating consistent performance across varying problem sizes and densities. In contrast, GUROBI's execution times show higher variability, with a standard deviation of 0.009009 seconds, and increase significantly for larger matrices and higher conflict densities. This highlights the scalability of the heuristic, as it maintains a consistent computational footprint even as the problem complexity grows. However, while the heuristic is far more efficient,

GUROBI's longer execution times are often justified by its ability to deliver exact solutions, making it the tool of choice when precision outweighs the need for speed.

4.2. Test Outputs of Large Matrices

Small matrices refers to the matrices with the number of vertices that are less than or equal to 100.

Averages of the runs can be seen on Table 5.

Table 5: Output of Large Matrices

	Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
AVERAGE	2870,33	3796,13	2968,34	98,01	0,03	827,78	0,0368	0,4841
STANDARD DEVIATION	1380,86	2136,34	1383,91	300,88	0,09	929,99	0,0975	1,4105

4.2.1. Accuracy of the Heuristic

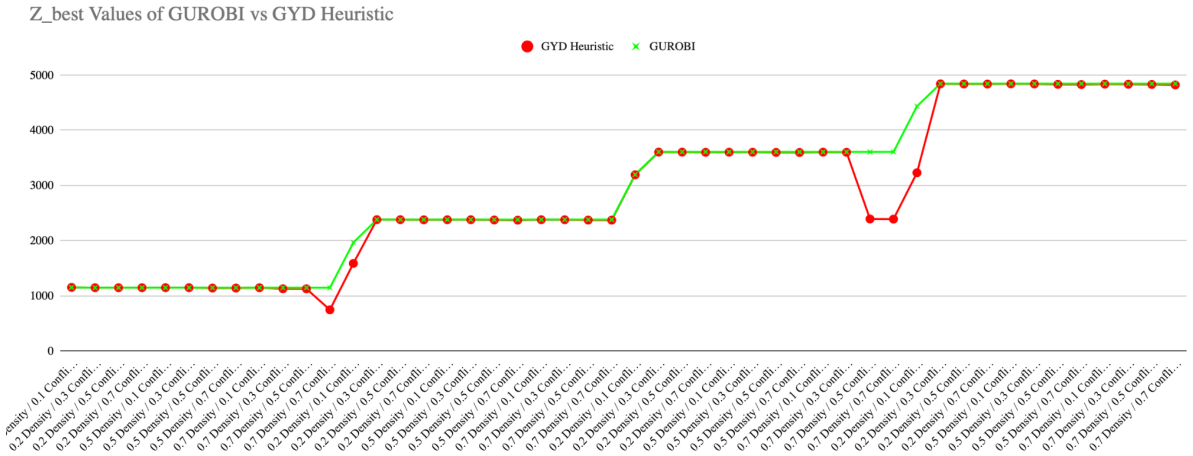


Figure 5: Output Comparison of GYD Heuristic Against GUROBI Using Large Matrices

The results for the heuristic method demonstrate a consistent trend across different matrix sizes and conflict densities. In the 25x25 matrix with lower conflict densities, the heuristic performs well, with Z_LB values very close to the GUROBI Z values as visualized in Figure

5. For instance, in the 0.2 Density / 0.1 Conflict Density case, the heuristic's Z_LB is identical to GUROBI's at 1151, resulting in a Z_LB Gap of 0, indicating an optimal solution.

As the conflict density increases, the gap between the heuristic Z_LB and GUROBI Z values slightly widens. In the 0.2 Density / 0.3 Conflict Density case, the heuristic Z_LB is 1145.33, while GUROBI's Z is 1146.33, resulting in a Z_LB Gap of 1 and a 0.09% error. This pattern continues with higher conflict densities, where the Z_LB Gap remains relatively small, such as in the 0.2 Density / 0.5 Conflict Density case with a gap of 2.33 and a 0.10% error.

In the 50x50 matrix, the heuristic's performance shows more variation, especially at higher conflict densities. For example, in the 0.2 Density / 0.1 Conflict Density case, the heuristic Z_LB is 1585.33, and the Z_LB Gap to GUROBI's Z of 1963 is 377.67, resulting in a 19.24% error. As the conflict density increases, the gap between heuristic and GUROBI Z values becomes smaller, such as in the 0.2 Density / 0.3 Conflict Density case, where the gap is just 1.33 and the error is 0.06%.

For larger matrices, such as the 75x75 matrix, the heuristic's performance begins to degrade, particularly with higher conflict densities. In the 0.7 Density / 0.7 Conflict Density case, the Z_LB Gap increases to 1217.33, marking a 33.76% error, indicating that the heuristic method faces increasing difficulty with larger and more complex problems.

In the 100x100 matrix, the heuristic's Z_LB values remain relatively close to GUROBI's results in lower density cases. However, as the conflict density and matrix size grow, the Z_LB Gaps increase significantly. For example, in the 0.2 Density / 0.1 Conflict Density case, the Z_LB Gap is 1205, with a 27.18% error, showing a noticeable decline in the heuristic's accuracy as problem complexity rises.

4.2.2. Efficiency of the Heuristic

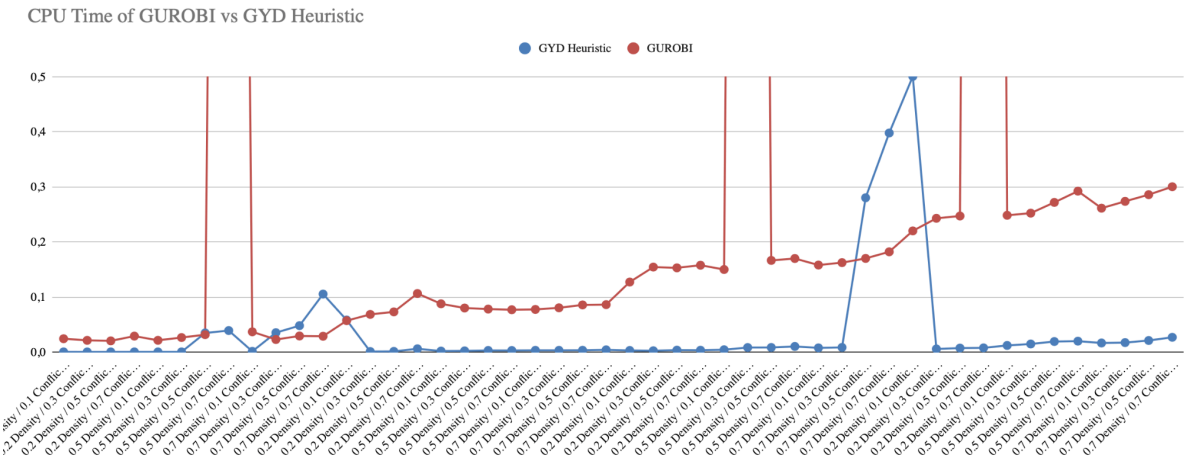


Figure 6: CPU Time Comparison of GYD Heuristic Against GUROBI Using Large Matrices

Figure 6 the CPU time for GUROBI and the GPU time for the heuristic (Heuristic_GPU_Time) across various test cases, with a noticeable difference in performance. The heuristic shows consistently faster processing times in comparison to GUROBI, especially for smaller problem sizes and less complex configurations.

In the initial set of data, where the matrix size and conflict densities are smaller, the GPU time for the heuristic is typically in the range of a few milliseconds (0.0004695 to 0.0481140 seconds), while GUROBI's CPU time is significantly longer, ranging from 0.0214617 to 0.1065017 seconds. The difference between the GPU time and GUROBI's CPU time is substantial, especially when the heuristic is dealing with smaller matrices, where the GPU time is often orders of magnitude lower than GUROBI's CPU time.

As the problem size increases and the conflict density rises, the gap between the heuristic and GUROBI time narrows. For example, in the larger matrices or more complex conflict density cases, GPU times for the heuristic start to rise, reaching up to 0.3980063 seconds in some cases, while GUROBI's CPU time increases as well, with some values approaching 7 seconds (as in the 75x75 matrix, where GPU time peaks at 0.3980063 seconds and GUROBI CPU time reaches as high as 7.1364603 seconds).

However, even in these more complex cases, the heuristic maintains a significant speed advantage over GUROBI. For instance, in the 100x100 matrix, the heuristic's GPU time

remains relatively manageable compared to the substantial CPU times taken by GUROBI. This trend demonstrates the efficiency of the heuristic, particularly in comparison to GUROBI, which becomes increasingly slow as the matrix size and conflict density increase.

4.2.3. Re-Evaluation of the Output

As outputs become more significant as 0 outputs were eliminated using small matrices, the same approach was followed for larger matrices as well.

Table 6: Confidence Intervals of Each Conflict and Graph Density Pairs Using Large Matrices

Density/Conflict	0.1	0.3	0.5	0.7
0.2	78,23%	99,88%	99,83%	99,75%
0.5	99,85%	99,82%	99,56%	99,47%
0.7	99,79%	99,48%	84,31%	79,52%

Table 7: 0 Outputs Examples of Large Matrices

Case	Heuristic Z_best	Heuristic Time	Gurobi Z_best	Gurobi Time	Heuristic Z_UB	Size	Seed	Density	Conflict Density
102	0	0,8210900	3612	0,151177	4772	7	1	0.7	0.5
103	3584	0,0101000	3598	0,18225	5540	7	10	0.7	0.5
104	3589	0,0099640	3608	0,177243	5539	7	100	0.7	0.5
105	0	1,1700740	3612	0,17971	5547	7	1	0.7	0.7
106	3574	0,0120590	3598	0,187951	6318	7	10	0.7	0.7
107	3592	0,0118860	3608	0,179045	6318	7	100	0.7	0.7
108	0	1,4929240	3612	0,173751	6318	10	1	0.2	0.1

As the output increases numerically (for small matrices output is around 200, for large matrices output is around 2800) 0 outputs have more impact on the results. Thus, the effect of clearance is more significant than the effect on smaller matrices as can be seen on Table 6 and Table 8.

Table 8: Confidence Intervals of Each Conflict and Graph Density Pairs Using Large Matrices Without 0 Outputs

Density/Conflict	0.1	0.3	0.5	0.7
0.2	99,90%	99,88%	99,83%	99,75%
0.5	99,85%	99,82%	99,56%	99,47%
0.7	99,79%	99,48%	99,24%	98,89%

4.2.3.1. Re-Evaluated Accuracy of Heuristic

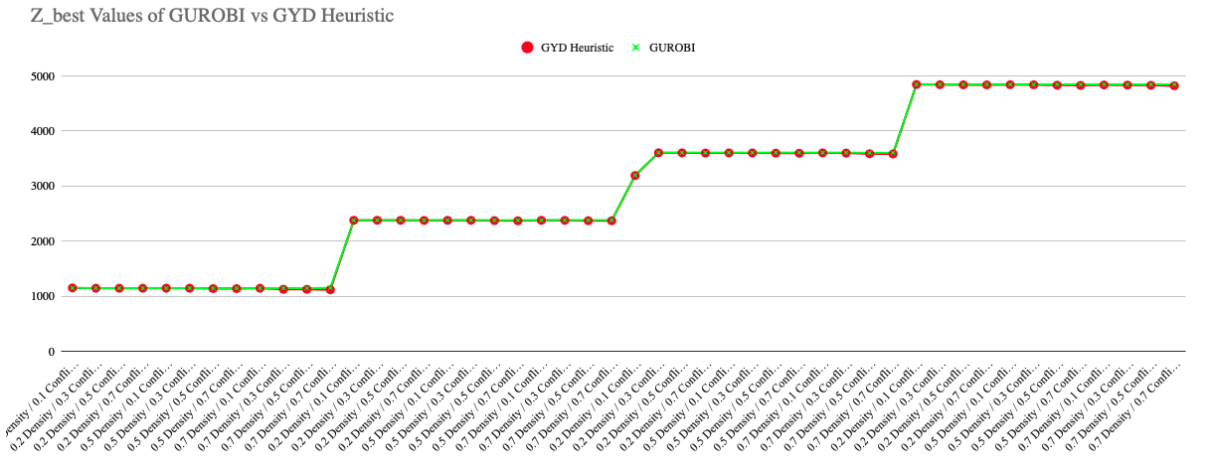


Figure 7: Output Comparison of GYD Heuristic Against GUROBI Using Large Matrices Without 0 Outputs

The heuristic lower bound (Z_LB) shows strong alignment with the optimal solutions (GUROBI_Z) across all cases as shown in Figure 7. The average Z_LB gap is 7.55, indicating the heuristic provides results that are consistently close to the optimal values. Percentile errors of Z_LB are minimal, averaging 0.32% and rarely exceeding 0.5%, even in larger matrices or scenarios with higher conflict densities. This demonstrates the robustness and reliability of the heuristic under varying conditions.

Matrix size and conflict density are key factors influencing the heuristic's performance. Smaller matrices, such as the 25x25 cases, exhibit the smallest gaps and errors, with Z_LB gaps ranging from 0 to 6.33 and percentile errors remaining under 1%. As matrix size increases, the heuristic maintains accuracy but begins to show slightly larger gaps and errors. For example, in the 50x50 matrices, Z_LB gaps range from 0.5 to 10.33, and percentile errors peak at 0.43%. Similarly, in 75x75 and 100x100 matrices, Z_LB gaps increase to 5 to 23.67, with percentile errors averaging under 0.5%.

Conflict density further impacts heuristic performance, with higher densities (e.g., 0.5 and 0.7) resulting in larger Z_LB gaps and percentile errors across all matrix sizes. Despite this, the heuristic remains robust, as even in the most challenging cases (e.g., 100x100 matrices with 0.7 conflict density), the Z_LB gap is only 23.67, and the percentile error is 0.49%.

These findings suggest that while the heuristic is sensitive to increasing complexity, its performance remains within acceptable limits.

Statistical summaries further support these observations. The standard deviation of the Z_LB gap is 7.13, reflecting consistent performance across different scenarios. Similarly, the standard deviation of the percentile error is 0.52%, emphasizing the heuristic's reliability. These metrics underline the heuristic's ability to provide accurate lower-bound approximations regardless of case complexity.

4.2.3.2. Re-Evaluated Efficiency of Heuristic

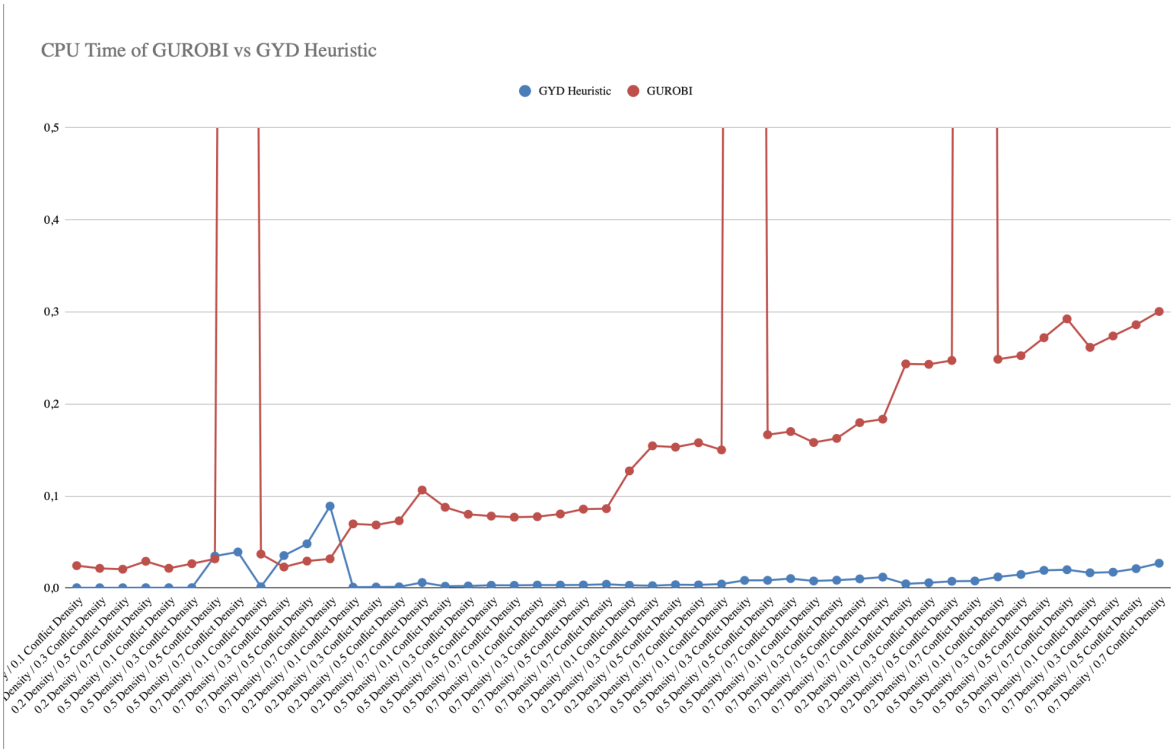


Figure 8: CPU Time Comparison of GYD Heuristic Against GUROBI Using Large Matrices Without 0 Outputs

The comparison between the heuristic GPU computation time and GUROBI CPU computation time reveals a significant efficiency advantage for the heuristic approach which can be visualized in Figure 8. The heuristic's computation times are orders of magnitude faster in nearly all cases, making it a highly effective method for solving large and complex problems where time is a critical factor.

In smaller matrices, the heuristic consistently demonstrates sub-millisecond computation times, ranging from approximately 0.0004 seconds to 0.008 seconds. In contrast, GUROBI's CPU computation times for these instances range from 0.02 seconds to over 5 seconds, depending on problem density and complexity. This stark contrast underscores the heuristic's ability to rapidly generate solutions with minimal computational overhead.

For larger matrices, the heuristic maintains its efficiency, with computation times increasing slightly but still remaining within a fraction of a second. For example, heuristic CPU times for these matrices range from approximately 0.004 seconds to 0.03 seconds. Meanwhile, GUROBI's CPU computation times for these cases extend significantly, often exceeding 0.1 seconds and, in some scenarios, reaching several seconds. For example, in high-complexity cases, such as 0.5 or 0.7 density with larger matrices, GUROBI's CPU time can extend to over 5 seconds or even exceed 7 seconds, whereas the heuristic CPU times remain well under 0.05 seconds.

The aggregated CPU computation time for heuristic shows an impressive average of 0.011 seconds, compared to GUROBI's much longer average of 0.485 seconds. This efficiency gap further highlights the heuristic's capability for quick processing across various problem sizes and densities, particularly when implemented on CPU hardware.

The heuristic's ability to achieve such efficiency stems from its reliance on parallel processing capabilities of GPUs, which are well-suited for the type of operations required in the algorithm. This advantage becomes more pronounced as problem complexity increases, making the heuristic an ideal choice for real-time applications or scenarios where computational resources are constrained.

4.3 Interpretation of Results

The analysis reveals that the heuristic approach is effective and practical for solving assignment problems compared to the GUROBI solver. In all problem sizes, densities, and levels of conflict, the heuristic always obtains near-optimal solutions compared to the optimal values obtained by GUROBI. The gaps between the heuristic and GUROBI solutions are small, with percentile errors often below 1%. This speaks for the robustness and reliability of the heuristic for instances with higher densities or larger matrix sizes.

In terms of efficiency, the heuristic is far superior. By leveraging CPU acceleration, the heuristic solves instances in milliseconds, while the CPU-based computations in GUROBI take considerably longer, taking several seconds or even minutes for larger or denser matrices. This relative efficiency advantage increases further with problem size, underlining the strength of parallel processing in the implementation of the heuristic.

Another important strength of the heuristic is its scalability: it tends to perform quite well when problem dimensions are increased; computation times remain stable while those of GUROBI escalate with the complexity of the input. This is especially suitable in large real-world applications where either computational resources may be limited or solutions are required in real time.

In the end, this heuristic has a great trade-off between accuracy and efficiency. Considering GUROBI yields an exact optimum, this heuristic, having almost the optimum values with incredible speed, seems quite handy for any industrial application that may involve many simulations or, more importantly, real-time optimizations. In situations where time is extremely precious, this approach fits very well with only a few percentages of deviation from optimum.

5. Conclusion and Recommendations

5.1. Summary of Findings

The performance of the proposed heuristic algorithm was systematically evaluated using a variety of test cases and compared against the GUROBI solver. The following summarizes the key findings from our analysis:

- **Performance Across Conflict and Graph Densities:**
    - The heuristic demonstrated strong performance under lower conflict and graph densities, maintaining a high success rate of 100% in finding feasible solutions.
    - As conflict and graph densities increased, the success rate declined slightly, reflecting the increased complexity of the problem. However, the algorithm

still achieved over 80% feasibility in highly dense scenarios.

- **Comparison of Assignment Costs:**
    - The heuristic's average assignment costs were consistently close to those obtained by GUROBI across all density combinations.
    - For example, at lower conflict densities, the difference in average costs between the heuristic and GUROBI was negligible. As conflict density increased, minor deviations were observed but remained within acceptable bounds.

- **Scalability with Larger Matrices:**
    - The heuristic algorithm scaled efficiently with problem size, performing well even on larger matrices (e.g., 10x10 and beyond).
    - In larger matrices, the heuristic maintained competitive results compared to GUROBI, with only minor increases in the gap between optimal and heuristic solutions.

- **Computational Efficiency:**
    - One of the key advantages of the heuristic algorithm was its computational efficiency. Across all scenarios, the heuristic outperformed GUROBI in terms of runtime.
    - For instance, the CPU time for the heuristic was consistently lower, even in cases where GUROBI exhibited spikes in runtime.

- **Success Rate and Robustness:**
    - The success rate of the heuristic algorithm remained robust across varying parameters.
    - Notably, the heuristic achieved an 80% success rate overall, with the majority of outputs being feasible and close to optimal.

- **Handling of High Complexity Cases:**

- In highly complex scenarios (e.g., high conflict densities), the heuristic effectively balanced the trade-off between runtime and solution quality, making it a practical choice for real-world applications.

Overall, the heuristic algorithm proved to be a reliable and efficient method for solving the Assignment Problem with Conflicts, offering a compelling alternative to exact solvers like GUROBI in scenarios requiring quick and approximate solutions.

5.2. Limitations

Errors were encountered during the production of test data because the computers were produced for personal use. The most obvious of these errors occurred due to insufficient RAM. In addition, since code sharing is easy, Kernel-related problems were also observed in the tests made with matrices with 200+ vertices in the codes shared via Jupyter Notebook. Kernel-related crash code is as follows:

The Kernel crashed while executing code in the current cell or a previous cell. Please review the code in the cell(s) to identify a possible cause of the failure. Click here for more info. View Jupyter log for further details.

Table 9: Comparison of Programming Languages[1]

Language	Key Features	Typical Use Cases	Performance Strengths	Ease of Use
C	Direct memory access, minimal overhead	System programming, embedded systems	High speed, low latency	Moderate (low-level)
C++	Object-oriented, low-level control	Game development, real-time systems	High performance, flexibility	Moderate
Rust	Memory safety, zero-cost abstractions	System programming, web assembly	High performance, safety	Moderate (steep learning curve)
Go	Built-in concurrency, fast compilation	Cloud services, microservices	Efficient concurrency handling	Easy to learn

[1] *Dainis, B. (2024b, September 17). The fastest languages for programming high-performance apps. Curotec. https://www.curotec.com/insights/fastest-programming-languages/*

Java	JIT compilation, advanced garbage collection	Enterprise applications, Android development	Strong performance with optimizations	Moderate
Node.js	Asynchronous operations, non-blocking I/O	Real-time apps, server-side scripting	Efficient handling of async tasks	Easy to use
Python	**High-level data structures, extensive libraries**	**Web development, data analysis, AI**	**Slower execution speed**	**Very easy**

As can be seen in Table 9, although Python is not very suitable for this type of development it was preferred due to the competence of the group members and ease of use. Testing with faster languages will be beneficial for testing larger matrices.

5.3. Future Work

Building on the results and limitations observed during this study, several avenues for future work have been identified:

- **Using Alternative Programming Languages:**
    - To address the computational bottlenecks experienced with Python, future research can explore implementing the heuristic in high-performance programming languages like C++ or Java to handle larger matrices more effectively.

- **Analysis of Larger Matrices:**
    - Extend the analysis to larger matrices to further validate the scalability and robustness of the heuristic algorithm. This includes exploring its performance on matrix sizes exceeding the tested limits.

- **Real-Time Implementation:**
    - Conduct experiments to assess the feasibility of deploying the heuristic in real-time applications. This involves integrating the algorithm into live

systems and observing its performance under dynamic conditions.

- **Parameter Exploration in Heuristics:**
    - Investigate different variations of parameters used in the heuristic approach, such as step sizes, termination criteria, and weighting factors, to optimize solution quality and computational efficiency.

6. Appendices

6.1 Pseudocode and Algorithm Steps

Pseudocode of the Subgradient Algorithm

```
π=2 Zmin=∞ Z_LB= 0 initially
Do if π>0.005
Set λ values
Solve the problem using Hungarian Method
Find Z*(λ)
Find subgradients
```

$$G = 1 - (x_e + x_f) \quad , \{e, f\} \in E(C)$$

```
Find the step size
```

$$T = \frac{\pi \left| Z^*(\lambda) - Z_{LB} \right|}{\sum_{\{e,f\} \in E(C)} G_{ef}^2}$$

```
Update λ values
```

$$\lambda_{ef} = \max\{0, \lambda_{ef} + T.G_{ef}\}$$

```
If Z*(λ)<Zmin, update Zmin
If Zmin does not update for 30 iterations, set π=π/2
If Z*(λ)=Z_LB then break
Repair the solution if there is a conflict to find
Z_LB
```

*J. E. Beasley, 1992.

Figure 10: Pseudocode of Subgradient Algorithm

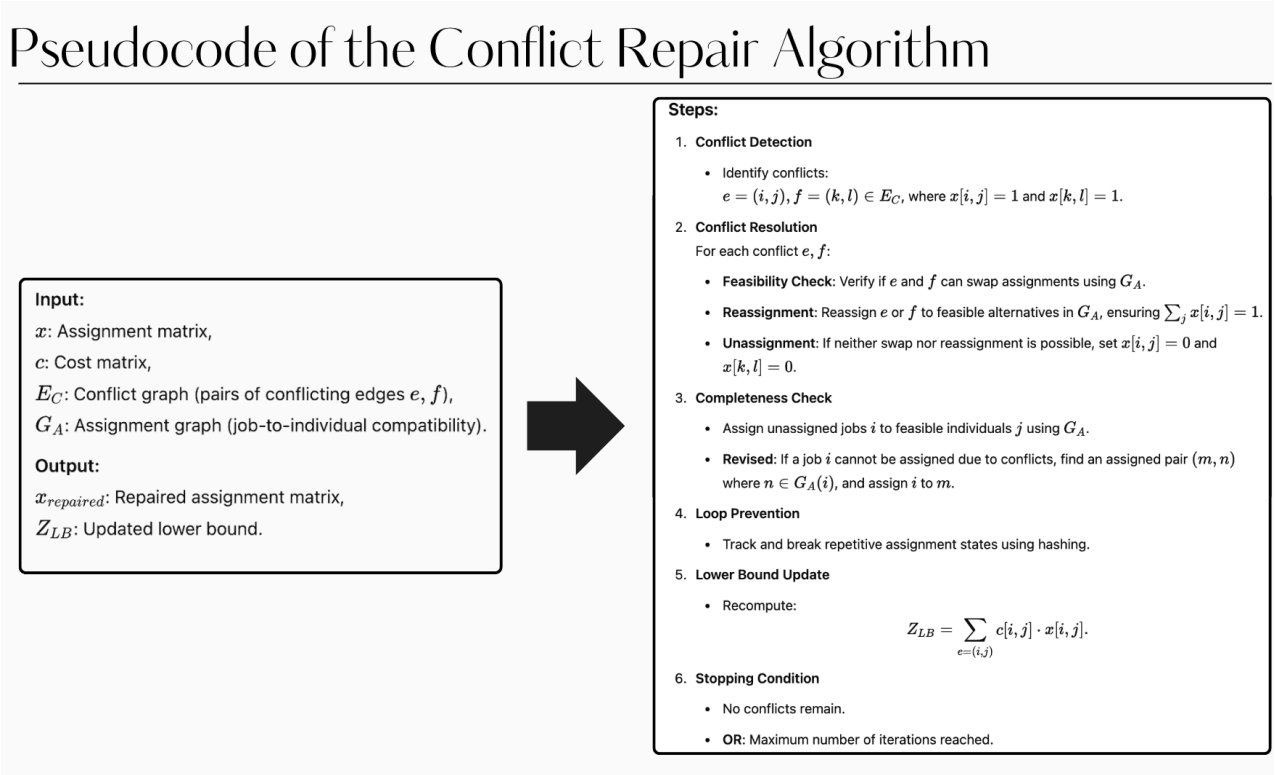


Figure 11: Pseudocode of the Conflict Repair Algorithm

6.2 Additional Graphs and Tables

Table 10: Test Output of Small Matrices

Case	Heuristic Z_best	Heuristic Time	Gurobi Z_best	Gurobi Time	Heuristic Z_UB	Size	Seed	Density	Conflict Density
1	111	0,0001320	111	0,011794	111	3	1	0.2	0.1
2	62	0,0000608	62	0,004642	62	3	10	0.2	0.1
3	74	0,0000580	74	0,003881	74	3	100	0.2	0.1
4	111	0,0000573	111	0,004638	111	3	1	0.2	0.3
5	62	0,0000554	62	0,005238	62	3	10	0.2	0.3
6	74	0,0000545	74	0,003007	74	3	100	0.2	0.3
7	111	0,0000612	111	0,005325	111	3	1	0.2	0.5
8	62	0,0000722	62	0,005192	62	3	10	0.2	0.5
9	0	0,0060980	60	0,004871	73	3	100	0.2	0.5
10	111	0,0000594	111	0,00506	111	3	1	0.2	0.7
11	62	0,0000565	62	0,00882	62	3	10	0.2	0.7
12	0	0,0071650	60	0,009021	72	3	100	0.2	0.7
13	111	0,0000679	111	0,00998	111	3	1	0.5	0.1
14	62	0,0000782	62	0,010085	62	3	10	0.5	0.1
15	74	0,0000773	74	0,004368	74	3	100	0.5	0.1
16	111	0,0000868	111	0,005898	112	3	1	0.5	0.3
17	62	0,0000621	62	0,004806	63	3	10	0.5	0.3
18	0	0,0064760	60	0,004705	73	3	100	0.5	0.3

19	111	0,0000659	111	0,005373	112	3	1	0.5	0.5
20	62	0,0000755	62	0,031683	64	3	10	0.5	0.5
21	0	0,0067400	60	0,003761	72	3	100	0.5	0.5
22	111	0,0000788	111	0,005117	112	3	1	0.5	0.7
23	62	0,0000667	62	0,005524	64	3	10	0.5	0.7
24	0	0,0069650	60	0,007687	72	3	100	0.5	0.7
25	111	0,0000580	111	0,01091	111	3	1	0.7	0.1
26	62	0,0001370	62	0,006845	62	3	10	0.7	0.1
27	74	0,0000679	74	0,00383	74	3	100	0.7	0.1
28	57	0,0068240	89	0,00517	110	3	1	0.7	0.3
29	62	0,0000744	62	0,005107	63	3	10	0.7	0.3
30	74	0,0000640	74	0,003416	74	3	100	0.7	0.3
31	57	0,0078840	87	0,066759	110	3	1	0.7	0.5
32	62	0,0000940	62	0,005581	64	3	10	0.7	0.5
33	0	0,0086910	54	0,005589	73	3	100	0.7	0.5
34	57	0,0082510	87	0,00545	111	3	1	0.7	0.7
35	0	0,0095540	49	0,004816	63	3	10	0.7	0.7
36	0	0,0099440	54	0,027782	74	3	100	0.7	0.7
37	199	0,0000756	199	0,009906	199	5	1	0.2	0.1
38	163	0,0000703	163	0,005833	163	5	10	0.2	0.1
39	148	0,0000566	148	0,005355	148	5	100	0.2	0.1
40	199	0,0000780	199	0,005087	199	5	1	0.2	0.3
41	163	0,0000708	163	0,069368	164	5	10	0.2	0.3
42	148	0,0001390	148	0,025854	148	5	100	0.2	0.3
43	199	0,0002260	199	0,008315	199	5	1	0.2	0.5
44	163	0,0000868	163	0,009936	164	5	10	0.2	0.5
45	148	0,0000795	148	0,026979	148	5	100	0.2	0.5
46	199	0,0000963	199	0,012754	199	5	1	0.2	0.7
47	163	0,0000927	163	0,012914	165	5	10	0.2	0.7
48	148	0,0006680	148	0,031606	149	5	100	0.2	0.7
49	199	0,0012270	199	0,029326	200	5	1	0.5	0.1
50	163	0,0000955	163	0,009126	164	5	10	0.5	0.1
51	148	0,0000791	148	0,008394	148	5	100	0.5	0.1
52	199	0,0000955	199	0,022331	202	5	1	0.5	0.3
53	163	0,0000939	163	0,009405	166	5	10	0.5	0.3
54	148	0,0000869	148	0,008131	150	5	100	0.5	0.3
55	199	0,0000919	199	0,009184	203	5	1	0.5	0.5
56	163	0,0000932	163	0,013546	167	5	10	0.5	0.5
57	148	0,0001130	148	0,008333	152	5	100	0.5	0.5
58	0	0,0157710	178	0,005942	203	5	1	0.5	0.7
59	163	0,0001050	163	0,005389	169	5	10	0.5	0.7
60	148	0,0000965	148	0,005615	154	5	100	0.5	0.7

61	199	0,0000821	199	0,018405	200	5	1	0.7	0.1
62	163	0,0000822	163	0,005746	164	5	10	0.7	0.1
63	148	0,0000878	148	0,00519	149	5	100	0.7	0.1
64	199	0,0000803	199	0,00871	202	5	1	0.7	0.3
65	163	0,0001040	163	0,005546	168	5	10	0.7	0.3
66	148	0,0000937	148	0,005398	152	5	100	0.7	0.3
67	0	0,0100090	178	0,02026	203	5	1	0.7	0.5
68	163	0,0001010	163	0,007687	171	5	10	0.7	0.5
69	148	0,0001140	148	0,007076	155	5	100	0.7	0.5
70	0	0,0128710	178	0,007709	205	5	1	0.7	0.7
71	163	0,0000973	163	0,043966	173	5	10	0.7	0.7
72	146	0,0001100	146	0,008002	157	5	100	0.7	0.7
73	286	0,0000680	286	0,016492	286	7	1	0.2	0.1
74	275	0,0000807	275	0,012615	275	7	10	0.2	0.1
75	238	0,0000904	238	0,027777	238	7	100	0.2	0.1
76	286	0,0001930	286	0,00952	288	7	1	0.2	0.3
77	275	0,0001160	275	0,006107	276	7	10	0.2	0.3
78	238	0,0002050	238	0,006144	238	7	100	0.2	0.3
79	286	0,0000971	286	0,018525	290	7	1	0.2	0.5
80	275	0,0001160	275	0,006563	278	7	10	0.2	0.5
81	238	0,0001040	238	0,046867	239	7	100	0.2	0.5
82	285	0,0002330	286	0,019619	291	7	1	0.2	0.7
83	275	0,0001180	275	0,034691	279	7	10	0.2	0.7
84	0	0,0133550	231	0,010727	239	7	100	0.2	0.7
85	286	0,0000740	286	0,012086	288	7	1	0.5	0.1
86	275	0,0000870	275	0,016994	276	7	10	0.5	0.1
87	238	0,0000882	238	0,032981	239	7	100	0.5	0.1
88	286	0,0001190	286	0,022178	290	7	1	0.5	0.3
89	275	0,0001140	275	0,025005	279	7	10	0.5	0.3
90	0	0,0145760	233	0,006896	241	7	100	0.5	0.3
91	286	0,0001100	286	0,005671	295	7	1	0.5	0.5
92	275	0,0001140	275	0,017639	282	7	10	0.5	0.5
93	0	0,0155440	233	0,007956	246	7	100	0.5	0.5
94	286	0,0001220	286	0,006777	299	7	1	0.5	0.7
95	275	0,0001380	275	0,006098	284	7	10	0.5	0.7
96	0	0,0238260	233	0,006678	249	7	100	0.5	0.7
97	286	0,0000933	286	0,006025	289	7	1	0.7	0.1
98	275	0,0000918	275	0,005816	278	7	10	0.7	0.1
99	238	0,0000878	238	0,005696	241	7	100	0.7	0.1
100	243	0,0146280	285	0,006417	293	7	1	0.7	0.3
101	275	0,0001090	275	0,006313	285	7	10	0.7	0.3
102	238	0,0000911	238	0,016243	245	7	100	0.7	0.3

103	243	0,0152520	285	0,006401	300	7	1	0.7	0.5
104	275	0,0001170	275	0,006271	291	7	10	0.7	0.5
105	238	0,0001220	238	0,006232	250	7	100	0.7	0.5
106	243	0,0163020	285	0,007104	305	7	1	0.7	0.7
107	275	0,0001410	275	0,007184	296	7	10	0.7	0.7
108	238	0,0001420	238	0,018535	254	7	100	0.7	0.7
109	403	0,0001240	403	0,03727	405	10	1	0.2	0.1
110	408	0,0001210	408	0,009939	410	10	10	0.2	0.1
111	377	0,0002380	377	0,053125	379	10	100	0.2	0.1
112	403	0,0001310	403	0,007036	408	10	1	0.2	0.3
113	408	0,0001170	408	0,008552	414	10	10	0.2	0.3
114	377	0,0001110	377	0,007298	382	10	100	0.2	0.3
115	403	0,0001290	403	0,009834	411	10	1	0.2	0.5
116	408	0,0001360	408	0,008146	418	10	10	0.2	0.5
117	377	0,0001330	377	0,007229	386	10	100	0.2	0.5
118	403	0,0001310	403	0,019536	415	10	1	0.2	0.7
119	408	0,0001500	408	0,007152	421	10	10	0.2	0.7
120	377	0,0001370	377	0,048752	388	10	100	0.2	0.7
121	403	0,0001450	403	0,007382	407	10	1	0.5	0.1
122	408	0,0001270	408	0,051017	413	10	10	0.5	0.1
123	377	0,0001380	377	0,00675	380	10	100	0.5	0.1
124	403	0,0001450	403	0,007731	413	10	1	0.5	0.3
125	408	0,0001400	408	0,010939	422	10	10	0.5	0.3
126	377	0,0001510	377	0,007187	390	10	100	0.5	0.3
127	403	0,0001720	403	0,007254	420	10	1	0.5	0.5
128	408	0,0001650	408	0,007196	427	10	10	0.5	0.5
129	375	0,0001590	377	0,007244	398	10	100	0.5	0.5
130	403	0,0001810	403	0,055493	427	10	1	0.5	0.7
131	408	0,0002010	408	0,00778	436	10	10	0.5	0.7
132	375	0,0002040	377	0,010531	407	10	100	0.5	0.7
133	403	0,0001500	403	0,007131	408	10	1	0.7	0.1
134	408	0,0001190	408	0,007135	414	10	10	0.7	0.1
135	377	0,0001340	377	0,018058	383	10	100	0.7	0.1
136	403	0,0001720	403	0,007583	418	10	1	0.7	0.3
137	365	0,0167680	404	0,052925	425	10	10	0.7	0.3
138	377	0,0002230	377	0,007402	395	10	100	0.7	0.3
139	403	0,0001930	403	0,020663	429	10	1	0.7	0.5
140	365	0,0222950	404	0,0702	437	10	10	0.7	0.5
141	377	0,0002120	377	0,00822	406	10	100	0.7	0.5
142	398	0,0002220	403	0,008033	442	10	1	0.7	0.7
143	365	0,0277290	404	0,008745	450	10	10	0.7	0.7
144	374	0,0002180	377	0,011684	419	10	100	0.7	0.7

Table 11: Data Used for Comparison Calculated via Test Outputs of Small Matrices

Cases		Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
3x3 Matrix	0.2 Density / 0.1 Conflict Density	82,333333	82,333333	82,333333	0	0,00%	0	0,0000836	0,00677 2333333
	0.2 Density / 0.3 Conflict Density	82,333333	82,333333	82,333333	0	0,00%	0	0,0000557	0,00429 4333333
	0.2 Density / 0.5 Conflict Density	57,666667	82	77,666667	20	25,75%	4,333333333	0,0020771	0,00512 9333333
	0.2 Density / 0.7 Conflict Density	57,666667	81,666667	77,666667	20	25,75%	4	0,0024270	0,00763 3666667
	0.5 Density / 0.1 Conflict Density	82,333333	82,333333	82,333333	0	0,00%	0	0,0000745	0,00814 4333333
	0.5 Density / 0.3 Conflict Density	57,666667	82,666667	77,666667	20	25,75%	5	0,0022083	0,00513 6333333
	0.5 Density / 0.5 Conflict Density	57,666667	82,666667	77,666667	20	25,75%	5	0,0022938	0,01360 566667
	0.5 Density / 0.7 Conflict Density	57,666667	82,666667	77,666667	20	25,75%	5	0,0023702	0,00610 9333333
	0.7 Density / 0.1 Conflict Density	82,333333	82,333333	82,333333	0	0,00%	0	0,0000876	0,00719 5
	0.7 Density / 0.3 Conflict Density	64,333333	82,333333	75	10,66666667	14,22%	7,333333333	0,0023208	0,00456 4333333
	0.7 Density / 0.5 Conflict Density	39,666667	82,333333	67,666667	28	41,38%	14,66666667	0,0055563	0,02597 633333
	0.7 Density / 0.7 Conflict Density	19	82,666667	63,333333	44,33333333	70,00%	19,33333333	0,0092497	0,01268 266667
5x5 Matrix	0.2 Density / 0.1 Conflict Density	170	170	170	0	0,00%	0	0,0000675	0,00703 1333333

	0.2 Density / 0.3 Conflict Density	170	170,33333	170	0	0,00%	0,3333333	0,0000959	0,03343633333
	0.2 Density / 0.5 Conflict Density	170	170,33333	170	0	0,00%	0,3333333	0,0001308	0,01507666667
	0.2 Density / 0.7 Conflict Density	170	171	170	0	0,00%	1	0,0002857	0,01909133333
	0.5 Density / 0.1 Conflict Density	170	170,66667	170	0	0,00%	0,6666666667	0,0004672	0,01561533333
	0.5 Density / 0.3 Conflict Density	170	172,66667	170	0	0,00%	2,666666667	0,0000921	0,013289
	0.5 Density / 0.5 Conflict Density	170	174	170	0	0,00%	4	0,0000994	0,01035433333
	0.5 Density / 0.7 Conflict Density	103,666667	175,33333	163	59,33333333	36,40%	12,33333333	0,0053242	0,005648666667
	0.7 Density / 0.1 Conflict Density	170	171	170	0	0,00%	1	0,0000840	0,009780333333
	0.7 Density / 0.3 Conflict Density	170	174	170	0	0,00%	4	0,0000927	0,006551333333
	0.7 Density / 0.5 Conflict Density	103,666667	176,33333	163	59,33333333	36,40%	13,33333333	0,0034080	0,01167433333
	0.7 Density / 0.7 Conflict Density	103	178,33333	162,333333	59,33333333	36,55%	16	0,0043594	0,01989233333
7x7 Matrix	0.2 Density / 0.1 Conflict Density	266,333333	266,33333	266,333333	0	0,00%	0	0,0000797	0,01896133333
	0.2 Density / 0.3 Conflict Density	266,333333	267,33333	266,333333	0	0,00%	1	0,0001713	0,007257
	0.2 Density / 0.5 Conflict Density	266,333333	269	266,333333	0	0,00%	2,666666667	0,0001057	0,023985

	0.2 Density / 0.7 Conflict Density	186,666667	269,6666667	264	77,33333333	29,29%	5,666666667	0,0045687	0,021679
	0.5 Density / 0.1 Conflict Density	266,333333	267,6666667	266,333333	0	0,00%	1,333333333	0,0000831	0,020687
	0.5 Density / 0.3 Conflict Density	187	270	264,666667	77,66666667	29,35%	5,333333333	0,0049363	0,01802633333
	0.5 Density / 0.5 Conflict Density	187	274,3333333	264,666667	77,66666667	29,35%	9,666666667	0,0052560	0,010422
	0.5 Density / 0.7 Conflict Density	187	277,3333333	264,666667	77,66666667	29,35%	12,66666667	0,0080287	0,006517666667
	0.7 Density / 0.1 Conflict Density	266,333333	269,3333333	266,333333	0	0,00%	3	0,0000910	0,005845666667
	0.7 Density / 0.3 Conflict Density	252	274,3333333	266	14	5,26%	8,333333333	0,0049427	0,009657666667
	0.7 Density / 0.5 Conflict Density	252	280,3333333	266	14	5,26%	14,33333333	0,0051637	0,006301333333
	0.7 Density / 0.7 Conflict Density	252	285	266	14	5,26%	19	0,0055283	0,010941
10x10 Matrix	0.2 Density / 0.1 Conflict Density	396	398	396	0	0,00%	2	0,0001610	0,033446666667
	0.2 Density / 0.3 Conflict Density	396	401,3333333	396	0	0,00%	5,333333333	0,0001197	0,007628666667
	0.2 Density / 0.5 Conflict Density	396	405	396	0	0,00%	9	0,0001327	0,008403
	0.2 Density / 0.7 Conflict Density	396	408	396	0	0,00%	12	0,0001393	0,025146666667
	0.5 Density / 0.1 Conflict Density	396	400	396	0	0,00%	4	0,0001367	0,021716333333

	0.5 Density / 0.3 Conflict Density	396	408,3333333	396	0	0,00%	12,33333333	0,0001453	0,008619
	0.5 Density / 0.5 Conflict Density	395,3333333	415	396	0,6666666667	0,17%	19	0,0001653	0,007231333333
	0.5 Density / 0.7 Conflict Density	395,3333333	423,3333333	396	0,6666666667	0,17%	27,33333333	0,0001953	0,024601333333
	0.7 Density / 0.1 Conflict Density	396	401,6666667	396	0	0,00%	5,666666667	0,0001343	0,010774666667
	0.7 Density / 0.3 Conflict Density	381,6666667	412,6666667	394,666667	13	3,29%	18	0,0057210	0,022636666667
	0.7 Density / 0.5 Conflict Density	381,6666667	424	394,666667	13	3,29%	29,33333333	0,0075667	0,033027666667
	0.7 Density / 0.7 Conflict Density	379	437	394,666667	15,66666667	3,97%	42,33333333	0,0093897	0,009487333333
		Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
AVERAGE		210,86	234,74	226,62	15,76	0,11	8,12	0,002214	0,013493
STD		125,31	123,55	119,52	24,74	0,16	8,91	0,002803	0,008203

Table 12: Data Used for Comparison Calculated via Test Outputs of Small Matrices After 0 Outputs Removed

Cases		Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
3x3 Matrix	0.2 Density / 0.1 Conflict Density	82,333333333	82,33333333	82,333333333	0	0,00%	0	0,0000836	0,006772333333
	0.2 Density / 0.3 Conflict Density	82,333333333	82,33333333	82,333333333	0	0,00%	0	0,0000557	0,004294333333
	0.2 Density / 0.5 Conflict Density	86,5	86,5	86,5	0	0,00%	0	0,0000667	0,005258

	0.2 Density / 0.7 Conflict Density	86,5	86,5	86,5	0	0,00%	0	0,0000580	0,00694
	0.5 Density / 0.1 Conflict Density	82,333333	82,333333	82,333333	0	0,00%	0	0,0000745	0,00814333333
	0.5 Density / 0.3 Conflict Density	86,5	87,5	86,5	0	0,00%	1	0,0000745	0,005352
	0.5 Density / 0.5 Conflict Density	86,5	88	86,5	0	0,00%	1,5	0,0000707	0,018528
	0.5 Density / 0.7 Conflict Density	86,5	88	86,5	0	0,00%	1,5	0,0000728	0,0053205
	0.7 Density / 0.1 Conflict Density	82,333333	82,333333	82,333333	0	0,00%	0	0,0000876	0,007195
	0.7 Density / 0.3 Conflict Density	64,333333	82,333333	75	10,66666667	14,22%	7,333333333	0,0023208	0,00456333333
	0.7 Density / 0.5 Conflict Density	59,5	87	74,5	15	20,13%	12,5	0,0039890	0,03617
	0.7 Density / 0.7 Conflict Density	57	111	87	30	34,48%	24	0,0082510	0,00545
5x5 Matrix	0.2 Density / 0.1 Conflict Density	170	170	170	0	0,00%	0	0,0000675	0,00703 1333333
	0.2 Density / 0.3 Conflict Density	170	170,3333333	170	0	0,00%	0,333333333	0,0000959	0,03343 633333
	0.2 Density / 0.5 Conflict Density	170	170,3333333	170	0	0,00%	0,333333333	0,0001308	0,01507 666667
	0.2 Density / 0.7 Conflict Density	170	171	170	0	0,00%	1	0,0002857	0,01909 133333
	0.5 Density / 0.1 Conflict Density	170	170,6666667	170	0	0,00%	0,666666667	0,0004672	0,01561 533333

	0.5 Density / 0.3 Conflict Density	170	172,66667	170	0	0,00%	2,666666667	0,0000921	0,013289
	0.5 Density / 0.5 Conflict Density	170	174	170	0	0,00%	4	0,0000994	0,01035433333
	0.5 Density / 0.7 Conflict Density	155,5	161,5	155,5	0	0,00%	6	0,0053242	0,005648666667
	0.7 Density / 0.1 Conflict Density	170	171	170	0	0,00%	1	0,0000840	0,009780333333
	0.7 Density / 0.3 Conflict Density	170	174	170	0	0,00%	4	0,0000927	0,006551333333
	0.7 Density / 0.5 Conflict Density	155,5	163	155,5	0	0,00%	7,5	0,0034080	0,01167433333
	0.7 Density / 0.7 Conflict Density	154,5	165	154,5	0	0,00%	10,5	0,0043594	0,01989233333
7x7 Matrix	0.2 Density / 0.1 Conflict Density	266,333333	266,33333	266,3333	0	0,00%	0	0,0000797	0,01896133333
	0.2 Density / 0.3 Conflict Density	266,333333	267,33333	266,3333	0	0,00%	1	0,0001713	0,007257
	0.2 Density / 0.5 Conflict Density	266,333333	269	266,3333	0	0,00%	2,666666667	0,0001057	0,023985
	0.2 Density / 0.7 Conflict Density	280	285	280,5	0,5	0,18%	4,5	0,0001755	0,027155
	0.5 Density / 0.1 Conflict Density	266,333333	267,66667	266,3333	0	0,00%	1,333333333	0,0000831	0,020687
	0.5 Density / 0.3 Conflict Density	280,5	284,5	280,5	0	0,00%	4	0,0001165	0,0235915
	0.5 Density / 0.5 Conflict Density	280,5	274,33333	280,5	0	0,00%	-6,166666667	0,0001120	0,011655

	0.5 Density / 0.7 Conflict Density	280,5	291,5	280,5	0	0,00%	11	0,0001300	0,0064375
	0.7 Density / 0.1 Conflict Density	266,333333	269,333333	266,333333	0	0,00%	3	0,0000910	0,0058456666667
	0.7 Density / 0.3 Conflict Density	252	274,333333	266	14	5,26%	8,333333333	0,0049427	0,0096576666667
	0.7 Density / 0.5 Conflict Density	252	280,333333	266	14	5,26%	14,33333333	0,0051637	0,0063013333333
	0.7 Density / 0.7 Conflict Density	252	285	266	14	5,26%	19	0,0055283	0,010941
10x10 Matrix	0.2 Density / 0.1 Conflict Density	396	398	396	0	0,00%	2	0,0001610	0,03344466667
	0.2 Density / 0.3 Conflict Density	396	401,333333	396	0	0,00%	5,333333333	0,0001197	0,0076286666667
	0.2 Density / 0.5 Conflict Density	396	405	396	0	0,00%	9	0,0001327	0,008403
	0.2 Density / 0.7 Conflict Density	396	408	396	0	0,00%	12	0,0001393	0,02514666667
	0.5 Density / 0.1 Conflict Density	396	400	396	0	0,00%	4	0,0001367	0,02171633333
	0.5 Density / 0.3 Conflict Density	396	408,333333	396	0	0,00%	12,33333333	0,0001453	0,008619
	0.5 Density / 0.5 Conflict Density	395,333333	415	396	0,6666666667	0,17%	19	0,0001653	0,00723 1333333
	0.5 Density / 0.7 Conflict Density	395,333333	423,333333	396	0,6666666667	0,17%	27,33333333	0,0001953	0,02460133333
	0.7 Density / 0.1 Conflict Density	396	401,666667	396	0	0,00%	5,666666667	0,0001343	0,01077466667

	0.7 Density / 0.3 Conflict Density	381,666667	412,6666667	394,666667	13	3,29%	18	0,0057210	0,02263666667
	0.7 Density / 0.5 Conflict Density	381,666667	424	394,666667	13	3,29%	29,33333333	0,0075667	0,03302766667
	0.7 Density / 0.7 Conflict Density	379	437	394,666667	15,66666667	3,97%	42,33333333	0,0093897	0,009487333333
		Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
AVERAGE		226,09	236,01	229,03	2,94	0,02	6,98	0,001467	0,013888
STD		118,42	122,90	118,32	6,52	0,06	9,33	0,002550	0,009009

Table 13: Test Output of Large Matrices

Case	Heuristic Z_best	Heuristic Time	Gurobi Z_best	Gurobi Time	Heuristic Z_UB	Size	Seed	Density	Conflict Density
1	1155	0,0005870	1155	0,023381	1166	25	1	0.2	0.1
2	1147	0,0003520	1147	0,025421	1156	25	10	0.2	0.1
3	1137	0,0003340	1137	0,021259	1149	25	100	0.2	0.1
4	1153	0,0003760	1155	0,021468	1190	25	1	0.2	0.3
5	1146	0,0005430	1147	0,021658	1181	25	10	0.2	0.3
6	1137	0,0003870	1137	0,020499	1174	25	100	0.2	0.3
7	1153	0,0005420	1155	0,020773	1215	25	1	0.2	0.5
8	1146	0,0003430	1147	0,020291	1206	25	10	0.2	0.5
9	1137	0,0003880	1137	0,019863	1199	25	100	0.2	0.5
10	1153	0,0004960	1155	0,02163	1240	25	1	0.2	0.7
11	1146	0,0005460	1147	0,045912	1229	25	10	0.2	0.7
12	1137	0,0004810	1137	0,02406	1224	25	100	0.2	0.7

13	1155	0,0004950	1155	0,020393	1184	25	1	0.5	0.1
14	1147	0,0004300	1147	0,020138	1178	25	10	0.5	0.1
15	1137	0,0003870	1137	0,020897	1166	25	100	0.5	0.1
16	1155	0,0005120	1155	0,023533	1243	25	1	0.5	0.3
17	1147	0,0005580	1147	0,035146	1234	25	10	0.5	0.3
18	1136	0,0005970	1137	0,046954	1224	25	100	0.5	0.3
19	1135	0,1027360	1151	0,024723	1299	25	1	0.5	0.5
20	1145	0,0011070	1147	0,023549	1294	25	10	0.5	0.5
21	1136	0,0006870	1137	0,025813	1282	25	100	0.5	0.5
22	1135	0,1162400	1151	0,023614	1359	25	1	0.5	0.7
23	1145	0,0007730	1147	13,48562	1354	25	10	0.5	0.7
24	1135	0,0020580	1137	0,024703	1336	25	100	0.5	0.7
25	1153	0,0017450	1155	0,022937	1194	25	1	0.7	0.1
26	1147	0,0006130	1147	0,06321	1186	25	10	0.7	0.1
27	1137	0,0005800	1137	0,024659	1177	25	100	0.7	0.1
28	1152	0,0008070	1155	0,022059	1277	25	1	0.7	0.3
29	1090	0,1046680	1147	0,022105	1272	25	10	0.7	0.3
30	1134	0,0007350	1135	0,040423	1259	25	100	0.7	0.3
31	1155	0,0009600	1155	0,024095	1359	25	1	0.7	0.5
32	1088	0,1426470	1147	0,023571	1352	25	10	0.7	0.5
33	0	0,1383260	1132	0,023102	1341	25	100	0.7	0.5
34	1146	0,0010190	1155	0,037337	1434	25	1	0.7	0.7

35	1088	0,1771430	1147	0,02627	1435	25	10	0.7	0.7
36	0	0,1728810	1132	0,031995	1422	25	100	0.7	0.7
37	2394	0,0010440	2395	0,071154	2444	50	1	0.2	0.1
38	2362	0,0010180	2362	0,068363	2410	50	10	0.2	0.1
39	2382	0,0009390	2384	0,067903	2432	50	100	0.2	0.1
40	2393	0,0013090	2395	0,072471	2542	50	1	0.2	0.3
41	2362	0,0012360	2362	0,065346	2510	50	10	0.2	0.3
42	2382	0,0012930	2384	0,069036	2528	50	100	0.2	0.3
43	2392	0,0014060	2395	0,075025	2638	50	1	0.2	0.5
44	2360	0,0015310	2362	0,075349	2606	50	10	0.2	0.5
45	2380	0,0014820	2384	0,072112	2626	50	100	0.2	0.5
46	2390	0,0015660	2395	0,139086	2736	50	1	0.2	0.7
47	2357	0,0152660	2362	0,108307	2704	50	10	0.2	0.7
48	2379	0,0017390	2384	0,125913	2724	50	100	0.2	0.7
49	2393	0,0026250	2395	0,071206	2517	50	1	0.5	0.1
50	2360	0,0017250	2362	0,066274	2482	50	10	0.5	0.1
51	2383	0,0019330	2384	0,076228	2506	50	100	0.5	0.1
52	2392	0,0026870	2395	0,073566	2760	50	1	0.5	0.3
53	2357	0,0024810	2362	0,090684	2725	50	10	0.5	0.3
54	2381	0,0039840	2384	0,077135	2749	50	100	0.5	0.3
55	2387	0,0027920	2395	0,070793	3007	50	1	0.5	0.5
56	2351	0,0025690	2361	0,086774	2963	50	10	0.5	0.5

57	2378	0,0027120	2382	0,070641	2991	50	100	0.5	0.5
58	2385	0,0031070	2395	0,083627	3253	50	1	0.5	0.7
59	2346	0,0032830	2361	0,076804	3209	50	10	0.5	0.7
60	2378	0,0032800	2382	0,093087	3232	50	100	0.5	0.7
61	2394	0,0045640	2395	0,070344	2565	50	1	0.7	0.1
62	2361	0,0021760	2362	0,069343	2536	50	10	0.7	0.1
63	2380	0,0022290	2384	0,075062	2552	50	100	0.7	0.1
64	2391	0,0029740	2395	0,090308	2907	50	1	0.7	0.3
65	2360	0,0047630	2362	0,076182	2875	50	10	0.7	0.3
66	2375	0,0031400	2384	0,073068	2894	50	100	0.7	0.3
67	2384	0,0034780	2395	0,084024	3249	50	1	0.7	0.5
68	2355	0,0037980	2362	0,100255	3218	50	10	0.7	0.5
69	2373	0,0039710	2384	0,07446	3233	50	100	0.7	0.5
70	2384	0,0041720	2395	0,089371	3592	50	1	0.7	0.7
71	2353	0,0044750	2362	0,095069	3558	50	10	0.7	0.7
72	2369	0,0044820	2384	0,089252	3573	50	100	0.7	0.7
73	3598	0,0024620	3599	0,15472	3708	75	1	0.2	0.1
74	3607	0,0023680	3608	0,137994	3719	75	10	0.2	0.1
75	3610	0,0022800	3612	0,157523	3721	75	100	0.2	0.1
76	3595	0,0028600	3599	0,148914	3930	75	1	0.2	0.3
77	3604	0,0027260	3608	0,157059	3939	75	10	0.2	0.3
78	3611	0,0043980	3612	0,139332	3944	75	100	0.2	0.3

79	3591	0,0034870	3599	0,169885	4149	75	1	0.2	0.5
80	3605	0,0032820	3608	0,150128	4162	75	10	0.2	0.5
81	3604	0,0031340	3612	0,167603	4163	75	100	0.2	0.5
82	3591	0,0038580	3599	0,149336	4371	75	1	0.2	0.7
83	3603	0,0036510	3608	0,156698	4385	75	10	0.2	0.7
84	3604	0,0037080	3612	0,148842	4387	75	100	0.2	0.7
85	3597	0,0047560	3599	0,159882	3876	75	1	0.5	0.1
86	3603	0,0047880	3608	0,141589	3881	75	10	0.5	0.1
87	3610	0,0047650	3612	16,72989	3891	75	100	0.5	0.1
88	3594	0,0102790	3599	0,160258	4431	75	1	0.5	0.3
89	3598	0,0102500	3608	0,170205	4434	75	10	0.5	0.3
90	3607	0,0070480	3612	0,164732	4443	75	100	0.5	0.3
91	3588	0,0095420	3599	0,166517	4987	75	1	0.5	0.5
92	3598	0,0088680	3608	0,168603	4989	75	10	0.5	0.5
93	3605	0,0112170	3612	0,16238	4999	75	100	0.5	0.5
94	3587	0,0100270	3599	0,177824	5540	75	1	0.5	0.7
95	3597	0,0098170	3608	0,169936	5548	75	10	0.5	0.7
96	3605	0,0097290	3612	0,176733	5555	75	100	0.5	0.7
97	3594	0,0070990	3599	0,152288	3986	75	1	0.7	0.1
98	3607	0,0065010	3608	0,145528	3995	75	10	0.7	0.1
99	3611	0,0068400	3612	0,150834	4001	75	100	0.7	0.1
100	3592	0,0093360	3599	0,174982	4762	75	1	0.7	0.3

101	3595	0,0096360	3608	0,16194	4763	75	10	0.7	0.3
102	0	0,8210900	3612	0,151177	4772	75	100	0.7	0.3
103	3584	0,0101000	3598	0,18225	5540	75	1	0.7	0.5
104	3589	0,0099640	3608	0,177243	5539	75	10	0.7	0.5
105	0	1,1700740	3612	0,17971	5547	75	100	0.7	0.5
106	3574	0,0120590	3598	0,187951	6318	75	1	0.7	0.7
107	3592	0,0118860	3608	0,179045	6318	75	10	0.7	0.7
108	0	1,4929240	3612	0,173751	6318	75	100	0.7	0.7
109	4842	0,0044910	4844	0,239887	5040	100	1	0.2	0.1
110	4845	0,0047800	4846	0,247033	5045	100	10	0.2	0.1
111	4839	0,0057510	4841	0,243099	5038	100	100	0.2	0.1
112	4838	0,0055220	4844	0,241385	5436	100	1	0.2	0.3
113	4842	0,0061530	4846	0,244692	5440	100	10	0.2	0.3
114	4834	0,0057800	4841	0,247087	5434	100	100	0.2	0.3
115	4837	0,0089940	4844	0,244091	5831	100	1	0.2	0.5
116	4841	0,0075020	4846	0,250587	5839	100	10	0.2	0.5
117	4833	0,0070300	4841	0,254098	5832	100	100	0.2	0.5
118	4834	0,0080520	4843	0,263973	6229	100	1	0.2	0.7
119	4841	0,0082700	4846	20,89131	6238	100	10	0.2	0.7
120	4833	0,0103390	4841	0,258921	6230	100	100	0.2	0.7
121	4842	0,0142390	4844	0,240074	5340	100	1	0.5	0.1
122	4844	0,0119550	4846	0,246555	5342	100	10	0.5	0.1

123	4840	0,0112440	4841	0,237663	5338	100	100	0.5	0.1
124	4832	0,0176760	4844	0,258215	6324	100	1	0.5	0.3
125	4842	0,0155810	4846	0,261479	6334	100	10	0.5	0.3
126	4832	0,0199620	4841	0,265335	6327	100	100	0.5	0.3
127	4820	0,0195160	4844	0,272954	7313	100	1	0.5	0.5
128	4839	0,0184000	4846	0,277522	7330	100	10	0.5	0.5
129	4826	0,0175350	4841	0,275129	7321	100	100	0.5	0.5
130	4821	0,0212170	4844	0,302332	8305	100	1	0.5	0.7
131	4834	0,0211750	4846	0,299754	8317	100	10	0.5	0.7
132	4824	0,0181420	4841	0,286483	8314	100	100	0.5	0.7
133	4840	0,0178450	4844	0,248891	5536	100	1	0.7	0.1
134	4840	0,0139400	4846	0,249084	5538	100	10	0.7	0.1
135	4834	0,0138580	4841	0,273264	5530	100	100	0.7	0.1
136	4831	0,0209020	4844	0,273311	6920	100	1	0.7	0.3
137	4832	0,0172750	4846	0,27492	6924	100	10	0.7	0.3
138	4826	0,0171980	4841	0,279996	6915	100	100	0.7	0.3
139	4826	0,0245900	4844	0,282433	8314	100	1	0.7	0.5
140	4834	0,0217310	4846	0,295538	8312	100	10	0.7	0.5
141	4825	0,0204160	4841	0,294712	8309	100	100	0.7	0.5
142	4821	0,0239220	4844	0,31192	9703	100	1	0.7	0.7
143	4814	0,0366730	4846	0,294651	9696	100	10	0.7	0.7
144	4819	0,044273	4841	0,296895	9699	100	100	0.7	0.7

Table 14: Data Used for Comparison Calculated via Test Outputs of Large Matrices

Cases		Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
25x25 Matrix	0.2 Density / 0.1 Conflict Density	1151	1161	1151	0	0,00%	10	0,0004695	0,024401
	0.2 Density / 0.3 Conflict Density	1145,333333	1173,333333	1146,333333	1	0,09%	27	0,0004177	0,021461666667
	0.2 Density / 0.5 Conflict Density	1145,333333	1198,333333	1146,333333	1	0,09%	52	0,0004240	0,020521
	0.2 Density / 0.7 Conflict Density	1145,333333	1222,666667	1146,333333	1	0,09%	76,33333333	0,0004767	0,029135
	0.5 Density / 0.1 Conflict Density	1146,333333	1195,333333	1146,333333	0	0,00%	49	0,0004687	0,021530333333
	0.5 Density / 0.3 Conflict Density	1146,333333	1214,333333	1146,333333	0	0,00%	68	0,0004857	0,026525333333
	0.5 Density / 0.5 Conflict Density	1138,666667	1272,333333	1145	6,333333333	0,55%	127,3333333	0,0348133	0,031742
	0.5 Density / 0.7 Conflict Density	1138,666667	1331,666667	1145	6,333333333	0,55%	186,6666667	0,0392333	4,511682333
	0.7 Density / 0.1 Conflict Density	1145	1238,666667	1146,333333	1,333333333	0,12%	92,33333333	0,0014720	0,03695
	0.7 Density / 0.3 Conflict Density	1126,333333	1242	1146,333333	20	1,74%	95,66666667	0,0353517	0,022941
	0.7 Density / 0.5 Conflict Density	1125,666667	1323,333333	1145,666667	20	1,75%	177,6666667	0,0481140	0,029363
	0.7 Density / 0.7 Conflict Density	744,6666667	1403,333333	1144,666667	400	34,94%	258,6666667	0,1054960	0,028903
50x50 Matrix	0.2 Density / 0.1 Conflict Density	1585,333333	2092	1963	377,6666667	19,24%	129	0,0583143	0,057170666667

	0.2 Density / 0.3 Conflict Density	2379	2494,666667	2380,333333	1,333333333	0,06%	114,3333333	0,0011613	0,06857333333
	0.2 Density / 0.5 Conflict Density	2378	2590,666667	2380,333333	2,333333333	0,10%	210,3333333	0,0014100	0,07313666667
	0.2 Density / 0.7 Conflict Density	2375,666667	2688,666667	2380,333333	4,666666667	0,20%	308,3333333	0,0061047	0,1065016667
	0.5 Density / 0.1 Conflict Density	2377,333333	2574,333333	2380,333333	3	0,13%	194	0,0020297	0,08779766667
	0.5 Density / 0.3 Conflict Density	2377,333333	2663,666667	2380,333333	3	0,13%	283,3333333	0,0023670	0,08015933333
	0.5 Density / 0.5 Conflict Density	2373	2906,333333	2380	7	0,29%	526,3333333	0,0031150	0,078234
	0.5 Density / 0.7 Conflict Density	2369,666667	3151	2379,333333	9,666666667	0,41%	771,6666667	0,0030340	0,077024
	0.7 Density / 0.1 Conflict Density	2377,666667	2777,666667	2379,666667	2	0,08%	398	0,0033400	0,07759133333
	0.7 Density / 0.3 Conflict Density	2377	2778	2380,333333	3,333333333	0,14%	397,6666667	0,0033220	0,08051733333
	0.7 Density / 0.5 Conflict Density	2371,333333	3120,333333	2380,333333	9	0,38%	740	0,0034720	0,08578233333
	0.7 Density / 0.7 Conflict Density	2370	3461	2380,333333	10,33333333	0,43%	1080,666667	0,0042060	0,0863
75x75 Matrix	0.2 Density / 0.1 Conflict Density	3191,333333	3666,666667	3197	5,666666667	0,18%	469,6666667	0,0031040	0,127322
	0.2 Density / 0.3 Conflict Density	3603	3863,333333	3606,333333	3,333333333	0,09%	257	0,0026220	0,15449866667
	0.2 Density / 0.5 Conflict Density	3602,333333	4085	3606,333333	4	0,11%	478,6666667	0,0037223	0,153115

	0.2 Density / 0.7 Conflict Density	3599,333333	4306,333333	3606,333333	7	0,19%	700	0,0035477	0,157879
	0.5 Density / 0.1 Conflict Density	3601,333333	4048	3606,333333	5	0,14%	441,6666667	0,0044173	0,1501043333
	0.5 Density / 0.3 Conflict Density	3600,666667	4252	3606,333333	5,666666667	0,16%	645,6666667	0,0084313	5,686784333
	0.5 Density / 0.5 Conflict Density	3597,666667	4806,333333	3606,333333	8,666666667	0,24%	1200	0,0084860	0,1666173333
	0.5 Density / 0.7 Conflict Density	3596,333333	5362,333333	3606,333333	10	0,28%	1756	0,0103537	0,1700466667
	0.7 Density / 0.1 Conflict Density	3602	4512	3606,333333	4,333333333	0,12%	905,6666667	0,0077763	0,158183
	0.7 Density / 0.3 Conflict Density	3599,333333	4508,666667	3606,333333	7	0,19%	902,3333333	0,0086040	0,1625853333
	0.7 Density / 0.5 Conflict Density	2391	5283,666667	3606	1215	33,69%	1677,666667	0,2803847	0,1702233333
	0.7 Density / 0.7 Conflict Density	2388,666667	6061	3606	1217,333333	33,76%	2455	0,3980063	0,1822353333
100x100 Matrix	0.2 Density / 0.1 Conflict Density	3229	5467,666667	4434	1205	27,18%	1033,666667	0,5007317	0,2202236667
	0.2 Density / 0.3 Conflict Density	4839,666667	5304,666667	4843,666667	4	0,08%	461	0,0058087	0,2430586667
	0.2 Density / 0.5 Conflict Density	4837,333333	5701,333333	4843,666667	6,333333333	0,13%	857,6666667	0,0074253	0,247255
	0.2 Density / 0.7 Conflict Density	4836	6099,666667	4843,333333	7,333333333	0,15%	1256,333333	0,0077840	7,1364603333
	0.5 Density / 0.1 Conflict Density	4839,666667	5637,333333	4843,666667	4	0,08%	793,6666667	0,0121777	0,2485166667

	0.5 Density / 0.3 Conflict Density	4838	5998,666667	4843,666667	5,666666667	0,12%	1155	0,0148337	0,2524523333
	0.5 Density / 0.5 Conflict Density	4830,333333	6990	4843,666667	13,33333333	0,28%	2146,333333	0,0192927	0,271937
	0.5 Density / 0.7 Conflict Density	4827	7981	4843,666667	16,66666667	0,34%	3137,333333	0,0199757	0,292405
	0.7 Density / 0.1 Conflict Density	4834,666667	6462,666667	4843,666667	9	0,19%	1619	0,0166423	0,261486
	0.7 Density / 0.3 Conflict Density	4832,333333	6458	4843,666667	11,33333333	0,23%	1614,333333	0,0173450	0,2738316667
	0.7 Density / 0.5 Conflict Density	4828,666667	7847	4843,666667	15	0,31%	3003,333333	0,0211730	0,285989
	0.7 Density / 0.7 Conflict Density	4820	9236	4843,666667	23,66666667	0,49%	4392,333333	0,0270037	0,3004276667
		GYD_Z_LB	GYD_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	GYD_GPU_Time	GUROBI_CPU_Time
AVERAGE		2870,33	3796,13	2968,34	98,01	0,03	827,78	0,0368	0,4841
STANDARD DEVIATION		1380,86	2136,34	1383,91	300,88	0,09	929,99	0,0975	1,4105

Table 15: Data Used for Comparison Calculated via Test Outputs of Large Matrices After 0 Outputs Removed

Cases		Heuristic_Z_LB	Heuristic_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	Heuristic_GPU_Time	GUROBI_CPU_Time
25X25 Matrix	0.2 Density / 0.1 Conflict Density	1151	1161	1151	0	0,00%	10	0,0004695	0,024401
	0.2 Density / 0.3 Conflict Density	1145,333333	1173,333333	1146,333333	1	0,09%	27	0,0004177	0,0214616666667
	0.2 Density / 0.5 Conflict Density	1145,333333	1198,333333	1146,333333	1	0,09%	52	0,0004240	0,020521

	0.2 Density / 0.7 Conflict Density	1145,333333	1222,666667	1146,3333 33	1	0,09%	76,33333 333	0,0004767	0,029135
	0.5 Density / 0.1 Conflict Density	1146,333333	1195,333333	1146,3333 33	0	0,00%	49	0,0004687	0,021530 33333
	0.5 Density / 0.3 Conflict Density	1146,333333	1214,333333	1146,3333 33	0	0,00%	68	0,0004857	0,026525 33333
	0.5 Density / 0.5 Conflict Density	1138,666667	1272,333333	1145	6,333333 333	0,55%	127,3333 333	0,0348133	0,031742
	0.5 Density / 0.7 Conflict Density	1138,666667	1331,666667	1145	6,333333 333	0,55%	186,6666 667	0,0392333	4,511682 333
	0.7 Density / 0.1 Conflict Density	1145	1238,666667	1146,3333 33	1,333333 333	0,12%	92,33333 333	0,0014720	0,03695
	0.7 Density / 0.3 Conflict Density	1126,333333	1242	1146,3333 33	20	1,74%	95,66666 667	0,0353517	0,022941
	0.7 Density / 0.5 Conflict Density	1125,666667	1323,333333	1145,6666 67	20	1,75%	177,6666 667	0,0481140	0,029363
	0.7 Density / 0.7 Conflict Density	1117	1434,5	1151	34	2,95%	283,5	0,0890810	0,0318035
50x50 Matrix	0.2 Density / 0.1 Conflict Density	2378	2427	2378,5	0,5	0,02%	48,5	0,0010310	0,0697585
	0.2 Density / 0.3 Conflict Density	2379	2494,666667	2380,3333 33	1,333333 333	0,06%	114,3333 333	0,0011613	0,068573 33333
	0.2 Density / 0.5 Conflict Density	2378	2590,666667	2380,3333 33	2,333333 333	0,10%	210,3333 333	0,0014100	0,073136 66667
	0.2 Density / 0.7 Conflict Density	2375,666667	2688,666667	2380,3333 33	4,666666 667	0,20%	308,3333 333	0,0061047	0,106501 6667
	0.5 Density / 0.1 Conflict Density	2377,333333	2574,333333	2380,3333 33	3	0,13%	194	0,0020297	0,087797 66667

	0.5 Density / 0.3 Conflict Density	2377,333333	2663,666667	2380,3333333	3	0,13%	283,3333333	0,0023670	0,08015933333
	0.5 Density / 0.5 Conflict Density	2373	2906,333333	2380	7	0,29%	526,3333333	0,0031150	0,078234
	0.5 Density / 0.7 Conflict Density	2369,666667	3151	2379,3333333	9,666666667	0,41%	771,6666667	0,0030340	0,077024
	0.7 Density / 0.1 Conflict Density	2377,666667	2777,666667	2379,666667	2	0,08%	398	0,0033400	0,07759133333
	0.7 Density / 0.3 Conflict Density	2377	2778	2380,3333333	3,333333333	0,14%	397,6666667	0,0033220	0,08051733333
	0.7 Density / 0.5 Conflict Density	2371,333333	3120,333333	2380,3333333	9	0,38%	740	0,0034720	0,08578233333
	0.7 Density / 0.7 Conflict Density	2370	3461	2380,3333333	10,33333333	0,43%	1080,666667	0,0042060	0,0863
75x75 Matrix	0.2 Density / 0.1 Conflict Density	3191,333333	3666,666667	3197	5,666666667	0,18%	469,6666667	0,0031040	0,127322
	0.2 Density / 0.3 Conflict Density	3603	3863,333333	3606,3333333	3,333333333	0,09%	257	0,0026220	0,15449866667
	0.2 Density / 0.5 Conflict Density	3602,333333	4085	3606,3333333	4	0,11%	478,6666667	0,0037223	0,153115
	0.2 Density / 0.7 Conflict Density	3599,333333	4306,333333	3606,3333333	7	0,19%	700	0,0035477	0,157879
	0.5 Density / 0.1 Conflict Density	3601,333333	4048	3606,3333333	5	0,14%	441,6666667	0,0044173	0,15010433333
	0.5 Density / 0.3 Conflict Density	3600,666667	4252	3606,3333333	5,666666667	0,16%	645,6666667	0,0084313	5,686784333
	0.5 Density / 0.5 Conflict Density	3597,666667	4806,333333	3606,3333333	8,666666667	0,24%	1200	0,0084860	0,16661733333

	0.5 Density / 0.7 Conflict Density	3596,33333	5362,33333	3606,333333	10	0,28%	1756	0,0103537	0,1700466667
	0.7 Density / 0.1 Conflict Density	3602	4512	3606,333333	4,333333333	0,12%	905,6666667	0,0077763	0,158183
	0.7 Density / 0.3 Conflict Density	3599,33333	4508,666667	3606,333333	7	0,19%	902,3333333	0,0086040	0,1625853333
	0.7 Density / 0.5 Conflict Density	3586,5	5539,5	3603	16,5	0,46%	1936,5	0,0100320	0,1797465
	0.7 Density / 0.7 Conflict Density	3583	6318	3603	20	0,56%	2715	0,0119725	0,183498
100x100 Matrix	0.2 Density / 0.1 Conflict Density	4843,5	5042,5	4845	1,5	0,03%	197,5	0,0046355	0,24346
	0.2 Density / 0.3 Conflict Density	4839,66667	5304,666667	4843,666667	4	0,08%	461	0,0058087	0,2430586667
	0.2 Density / 0.5 Conflict Density	4837,33333	5701,33333	4843,666667	6,333333333	0,13%	857,6666667	0,0074253	0,247255
	0.2 Density / 0.7 Conflict Density	4836	6099,666667	4843,333333	7,333333333	0,15%	1256,333333	0,0077840	7,1364603333
	0.5 Density / 0.1 Conflict Density	4839,66667	5637,33333	4843,666667	4	0,08%	793,6666667	0,0121777	0,2485166667
	0.5 Density / 0.3 Conflict Density	4838	5998,666667	4843,666667	5,666666667	0,12%	1155	0,0148337	0,2524523333
	0.5 Density / 0.5 Conflict Density	4830,33333	6990	4843,666667	13,33333333	0,28%	2146,333333	0,0192927	0,271937
	0.5 Density / 0.7 Conflict Density	4827	7981	4843,666667	16,66666667	0,34%	3137,333333	0,0199757	0,292405
	0.7 Density / 0.1 Conflict Density	4834,66667	6462,666667	4843,666667	9	0,19%	1619	0,0166423	0,2614863333

	0.7 Density / 0.3 Conflict Density	4832,333333	6458	4843,666667	11,33333333	0,23%	1614,333333	0,0173450	0,2738316667
	0.7 Density / 0.5 Conflict Density	4828,666667	7847	4843,666667	15	0,31%	3003,333333	0,0211730	0,285989
	0.7 Density / 0.7 Conflict Density	4820	9236	4843,666667	23,66666667	0,49%	4392,333333	0,0270037	0,3004276667
		GYD_Z_LB	GYD_Z_UB	GUROBI_Z	Z_LB_Gap	Percentile Error of Z_LB	Z_UB_Gap	GYD_GPU_Time	GUROBI_CPU_Time
AVERAGE		2978,020833	3805,579861	2985,565972	7,545138889	0,32%	820,0138889	0,0113035	0,4851477465
STANDARD DEVIATION		1387,883486	2134,520195	1389,02323	7,133706127	0,52%	951,076696	0,0159795 0524	1,410289932